

UNIVERSITATEA “LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE ȘI INGINERIE ELECTRICĂ

PROIECT DE DIPLOMĂ

Conducător științific: Șef lucr. Alexandru DOROBANȚIU
Îndrumător: Șef lucr. Alexandru DOROBANȚIU

Absolvent:
Răzvan-Bucur HOADREA
Specializarea CALCULATOARE

UNIVERSITATEA “LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE ȘI INGINERIE ELECTRICĂ

TITLUL TEMEI

Comparare algoritmi de predicție cu învățare automată

Conducător științific: Șef lucr. Alexandru DOROBANȚIU

Îndrumător: Șef lucr. Alexandru DOROBANȚIU

Absolvent:
Răzvan-Bucur HOADREA
Specializarea CALCULATOARE

UNIVERSITATEA "LUCIAN BLAGA" DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE
ȘI INGINERIE ELECTRICĂ

APROBAT (data)

DIRECTOR DEPARTAMENT

PLAN TEMATIC
pentru proiectul de diplomă

Proiectul de diplomă dat studentului: Răzvan-Bucur HOADREA

- 1. Tema proiectului:** Comparare algoritmi de predicție cu învățare automată
- 2. Termenul de predare a proiectului:** Iunie 2026
- 3. Elemente inițiale pentru proiect:** Învățare automată, Benchmark competiție sportivă
- 4. Nota explicativă (enumerarea problemelor care vor fi rezolvate):**
 - Implementarea unui model de învățare automată
 - Aplicare de predicții pe baza seturilor de date
 - Efectuarea unui studiu comparativ între mai mulți algoritmi de învățare automată
- 5. Enumerarea materialului grafic (cu indicarea precisă a desenelor obligatorii):**
 - Schema arhitecturii algoritmilor de învățare automată
 - Graficul comparativ al performanței algoritmilor
- 6. Consultații pentru proiect (săptămânal):** MIERCURI 14:00–15:00
- 7. Data eliberării temei:** OCTOMBRIE 2025.

Tema a fost primită pentru îndeplinire:
Data 31.10.2025

CONDUCĂTOR,
(grad didactic, prenume, nume)

Semnătură student

Șef lucr. Alexandru DOROBANȚIU

(semnătura)

Cuprins

1	Introducere	5
1.1	Prezentarea temei	5
1.2	Motivație	6
1.3	Scop și obiective	6
1.4	Structura lucrării	6
2	Considerații teoretice	8
2.1	Învățarea automată: paradigme și definiții	8
2.2	Clasificarea supervizată și formularea problemei	9
2.3	Compromisul bias–varianță, supraînvățarea și regularizarea	10
2.4	Problema dezechilibrului de clase	10
2.5	Regresia logistică	11
2.5.1	Modelul și interpretarea log-odds	11
2.5.2	Estimarea prin maximul verosimilității și entropia încrucișată	12
2.5.3	Coborârea pe gradient	12
2.5.4	Regularizarea și standardizarea	13
2.6	Arbori de decizie	13
2.6.1	Partiționarea recursivă	13
2.6.2	Măsuri de impuritate	14

2.6.3	Criterii de oprire, supraînvățare și instabilitate	15
2.7	Pădurea aleatoare (Random Forest)	15
2.7.1	Justificarea teoretică a ansamblurilor	15
2.7.2	Bagging și pădurea aleatoare	15
2.7.3	Estimarea out-of-bag și importanța caracteristicilor	16
2.7.4	Alte metode de ansamblu	17
2.8	Ingineria caracteristicilor și scurgerea de informație	17
2.9	Validarea și estimarea performanței	18
2.10	Metrici de evaluare	18
2.11	Calibrarea probabilităților	20
2.12	Sinteză comparativă a algoritmilor studiați	20
2.13	Lucrări conexe	20
3	Rezolvarea temei de proiect	22
3.1	Arhitectura sistemului	22
3.2	Tehnologii folosite și organizarea codului	23
3.3	Sursa de date	24
3.4	Construirea setului de date	24
3.5	Analiza exploratorie a setului de date	25
3.6	Ingineria caracteristicilor cauzale	26
3.6.1	Principiul cauzal și starea istorică	27
3.6.2	Valori implicite pentru piloți și echipe fără istoric	28
3.6.3	Inventarul complet al caracteristicilor	28
3.6.4	Eticheta și matricea de caracteristici	28
3.7	Implementarea modelelor	28

3.7.1	Contractul comun și standardizarea	30
3.7.2	Regresia logistică	30
3.7.3	Arborele de decizie	31
3.7.4	Pădurea aleatoare	32
3.7.5	Complexitatea computațională	33
3.7.6	Fabrica de modele și hiperparametrii	34
3.8	Predicția podiumului și agregarea în campionat	34
3.8.1	Ordonarea piloților per cursă	34
3.8.2	Agregarea în clasament de campionat	35
3.9	Validarea temporală pe ani	36
3.10	Antrenarea incrementală în-sezon	37
3.11	Metrici și instrumentația experimentală	37
3.12	Interfața în linie de comandă	38
3.12.1	Fluxul de lucru complet	38
3.13	Testarea automată și verificarea corectitudinii	39
3.14	Reproductibilitate	40
3.15	Decizii de proiectare și compromisuri	40
4	Rezultate	42
4.1	Configurația experimentală	42
4.2	Performanța comparativă	42
4.3	Variația performanței pe sezoane	43
4.4	Costul computațional al antrenării	44
4.5	Discuție: acuratețea în prezența dezechilibrului	45
4.6	Importanța caracteristicilor	45

4.7	Efectul datelor in-sezon	46
4.8	Sinteza rezultatelor	48
5	Concluzii și dezvoltări ulterioare	49
5.1	Concluzii generale	49
5.2	Gradul de îndeplinire a obiectivelor	49
5.3	Concluzii privind studiul comparativ	49
5.4	Contribuții personale	50
5.5	Dezvoltări ulterioare	51
	Bibliografie	52

Capitolul 1

Introducere

1.1 Prezentarea temei

Predicția rezultatelor sportive este un domeniu de aplicație clasic pentru învățarea automată, deoarece combină date structurate, abundente și actualizate periodic cu o problemă de decizie clar definită. Formula 1 oferă un cadru deosebit de potrivit: fiecare sezon generează zeci de curse, fiecare cursă produce rezultate pentru aproximativ douăzeci de piloți, iar regulile competiției sunt stabile pe perioade de mai mulți ani. Spre deosebire de sporturile de echipă, în care rezultatul depinde de interacțiuni complexe între mulți jucători, în Formula 1 performanța individuală a pilotului și a echipei poate fi urmărită numeric de la o cursă la alta.

Lucrarea de față abordează problema predicției *podiumului* unei curse de Formula 1, adică a identificării celor trei piloți care vor termina pe primele trei locuri (P1, P2 și P3). Problema este formulată ca o sarcină de clasificare binară: pentru fiecare pereche (cursă, pilot) se estimează probabilitatea ca pilotul respectiv să termine pe podium, după care, pentru fiecare cursă, piloții sunt ordonați descrescător după această probabilitate, iar primii trei formează podiumul prezis. Pe lângă predicția per cursă, probabilitățile sunt agregate într-un clasament de campionat estimat la finalul sezonului.

Elementul central al lucrării nu este atât aplicația în sine, cât **studiul comparativ** al trei algoritmi de învățare automată implementați integral, fără utilizarea unor biblioteci de învățare automată gata făcute: regresia logistică, arborele de decizie și pădurea aleatoare (*Random Forest*). Implementarea de la zero, peste biblioteca de calcul numeric NumPy [1], urmărește înțelegerea în profunzime a mecanismelor fiecărui algoritm și controlul complet asupra procesului de antrenare și evaluare.

1.2 Motivație

Alegerea temei este motivată de trei aspecte. În primul rând, implementarea de la zero a algoritmilor expune detaliile pe care bibliotecile consacrate, precum scikit-learn [2], le ascund: coborârea pe gradient, criteriul de divizare bazat pe entropie sau mecanismul de *bagging*. În al doilea rând, datele din Formula 1 ridică o provocare metodologică reală — *scurgerea de informație* (eng. *data leakage*) —, deoarece orice caracteristică folosită pentru a prezice o cursă trebuie calculată exclusiv din rezultate anterioare acelei curse. În al treilea rând, distribuția claselor este puternic dezechilibrată (doar aproximativ 15% dintre piloți termină pe podium), ceea ce face ca metrica de acuratețe globală să fie înșelătoare [3], [4] și impune folosirea unor metrice adecvate.

1.3 Scop și obiective

Scopul lucrării este realizarea unui studiu comparativ riguros între trei algoritmi de învățare automată aplicați aceleiași probleme de predicție, în condiții identice de antrenare și evaluare. Obiectivele concrete, în acord cu planul tematic al proiectului, sunt următoarele:

1. **Implementarea de la zero** a trei modele de învățare automată (regresie logistică, arbore de decizie și pădure aleatoare), folosind doar NumPy pentru algebra vectorizată.
2. **Aplicarea predicțiilor** pe un set de date real, alcătuit din rezultatele curselor de Formula 1 din sezoanele 2018–2025, cu o etapă de inginerie a caracteristicilor strict cauzală, fără scurgere de informație.
3. **Efectuarea unui studiu comparativ** între cei trei algoritmi din punctul de vedere al calității predicțiilor, măsurată prin patru metrice: acuratețe, rata de nimerire a podiumului, podiumul exact și acuratețea câștigătorului (Top-1).
4. **Evaluarea cu separare temporală în antrenare, validare și test pe ani**, pentru o estimare corectă a performanței, fără scurgere de informație.
5. **Analiza efectului datelor in-sezon**, prin compararea predicțiilor obținute cu și fără cursele deja disputate din sezonul curent, pentru a evalua câștigul antrenării incrementale.

1.4 Structura lucrării

Lucrarea este organizată în cinci capitole. Capitolul 2 prezintă fundamentele teoretice ale învățării automate și ale celor trei algoritmi studiați, împreună cu metodele de validare și metricele de evaluare. Capitolul 3 descrie soluția dezvoltată: arhitectura sistemului, sursa de date, ingineria caracteristicilor

cauzale, implementarea algoritmilor și a procedurii de validare. Capitolul 4 prezintă și interpretează rezultatele experimentale. Capitolul 5 sintetizează concluziile și propune direcții de dezvoltare ulterioară.

Capitolul 2

Considerații teoretice

Acest capitol prezintă sinteza cunoștințelor actuale din domeniul învățării automate aplicate predicției și fundamentele teoretice ale metodelor folosite în dezvoltarea temei. Expunerea pornește de la cadrul general al învățării automate și al clasificării supervizate (Secțiunile 2.1–2.3), continuă cu o problemă metodologică centrală pentru datele studiate — dezechilibrul de clase (Secțiunea 2.4) — și detaliază apoi bazele teoretice ale celor trei algoritmi implementați: regresia logistică (Secțiunea 2.5), arborele de decizie (Secțiunea 2.6) și pădurea aleatoare (Secțiunea 2.7). Ultimele secțiuni tratează aspectele transversale ale oricărui studiu experimental riguros: ingineria caracteristicilor și scurgerea de informație (Secțiunea 2.8), validarea și estimarea performanței (Secțiunea 2.9), metricile de evaluare (Secțiunea 2.10), calibrarea probabilităților (Secțiunea 2.11) și, în final, o trecere în revistă a lucrărilor conexe (Secțiunea 2.13).

2.1 Învățarea automată: paradigme și definiții

Învățarea automată (eng. *machine learning*) este subdomeniul inteligenței artificiale care studiază algoritmi capabili să își îmbunătățească performanța la o sarcină pe baza experienței, fără a fi programați explicit pentru fiecare situație [5], [6]. O definiție operațională larg citată este cea a lui Mitchell [5]: un program învață din experiența E , în raport cu o sarcină T și o măsură de performanță P , dacă performanța sa la T , măsurată prin P , se îmbunătățește odată cu E . În cazul de față, sarcina T este predicția podiumului unei curse, experiența E este mulțimea rezultatelor istorice ale curselor, iar performanța P este măsurată prin metricile descrise în Secțiunea 2.10.

În funcție de natura experienței, se disting în mod tradițional trei paradigme [6], [7]. În *învățarea supervizată*, fiecare exemplu de antrenament este însoțit de o etichetă-țintă, iar algoritmul învață asocierea dintre intrare și etichetă. În *învățarea nesupervizată*, datele nu au etichete, iar scopul este descoperirea unor structuri latente (grupări, reducerea dimensionalității). În *învățarea cu întărire* (eng. *reinforcement learning*), un agent învață o politică de acțiune din recompense obținute prin interacțiu-

nea cu un mediu. Problema abordată în această lucrare se încadrează în prima paradigmă: dispunem de rezultate istorice etichetate (un pilot a terminat sau nu pe podium) și dorim să prezicem eticheta pentru curse viitoare.

Învățarea supervizată se subîmparte, la rândul ei, după tipul etichetei. Când eticheta este o valoare numerică continuă, vorbim despre *regresie*; când eticheta aparține unei mulțimi finite de categorii, vorbim despre *clasificare* [8]. Formal, algoritmul primește un set de exemple de antrenament $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, unde $\mathbf{x}_i \in \mathbb{R}^d$ este un vector de d caracteristici (eng. *features*), iar $y_i \in \mathcal{Y}$ este eticheta asociată. Obiectivul este învățarea unei funcții $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ care nu doar reproduce etichetele de antrenament, ci *generalizează* bine pe date nevăzute.

Procesul de învățare este formulat ca minimizarea unui *risc*. Dacă $\ell(f(\mathbf{x}), y)$ este o funcție de pierdere care penalizează discrepanța dintre predicție și etichetă, riscul real (de generalizare) este media pierderii pe distribuția necunoscută a datelor, $R(f) = \mathbb{E}_{(\mathbf{x}, y)}[\ell(f(\mathbf{x}), y)]$. Deoarece distribuția este necunoscută, în practică se minimizează *riscul empiric*, estimat pe setul de antrenament [7]:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i). \quad (2.1)$$

Diferența dintre riscul empiric și riscul real este sursa fenomenului de supraînvățare, discutat în Secțiunea 2.3: un model poate atinge un risc empiric foarte mic, memorând particularitățile datelor de antrenament, fără ca aceasta să garanteze un risc real scăzut.

2.2 Clasificarea supervizată și formularea problemei

Atunci când eticheta ia exact două valori, $\mathcal{Y} = \{0, 1\}$, problema se numește *clasificare binară*. În lucrarea de față, eticheta $y = 1$ semnifică faptul că pilotul termină pe podium (locul ≤ 3), iar $y = 0$ în caz contrar. Un clasificator poate fi *discriminativ*, modelând direct granița de decizie sau probabilitatea condiționată $P(y | \mathbf{x})$, ori *generativ*, modelând distribuția comună $P(\mathbf{x}, y)$ [6]. Toate cele trei modele studiate aici sunt discriminative.

Multe clasificatoare nu produc direct eticheta, ci o *probabilitate* $\hat{p} = P(y = 1 | \mathbf{x}) \in [0, 1]$, transformată ulterior într-o etichetă printr-un prag de decizie (uzual 0,5): se prezice clasa pozitivă dacă $\hat{p} \geq 0,5$. Această probabilitate este elementul-cheie al soluției propuse: în loc de a fi binarizată pe fiecare pilot independent, ea servește drept *scor de ordonare* a piloților în cadrul fiecărei curse. Astfel, problema de clasificare este reîncadrată ca o problemă de *ordonare* (eng. *learning to rank*) [9]: pentru o cursă dată, piloții sunt sortați descrescător după $\hat{p}$, iar primii trei formează podiumul prezis. Această reformulare este importantă deoarece un clasificator binar aplicat independent nu garantează exact trei predicții pozitive per cursă, în timp ce ordonarea produce întotdeauna un podium bine definit.

2.3 Compromisul bias–varianță, supraînvățarea și regularizarea

Capacitatea de generalizare a unui model este guvernată de *compromisul bias–varianță* (eng. *bias–variance tradeoff*) [7], [10]. Pentru o pierdere pătratică, eroarea de generalizare așteptată a unui model într-un punct se descompune în trei termeni: pătratul *bias*-ului (eroarea sistematică introdusă de ipotezele simplificatoare ale modelului), *varianța* (sensibilitatea modelului la fluctuațiile setului de antrenament) și un *zgomot* ireductibil, propriu datelor:

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \underbrace{(\text{Bias}[\hat{f}(\mathbf{x})])^2}_{\text{eroare sistematică}} + \underbrace{\text{Var}[\hat{f}(\mathbf{x})]}_{\text{instabilitate}} + \underbrace{\sigma^2}_{\text{zgomot}}. \quad (2.2)$$

Un model prea simplu (de exemplu, o graniță liniară pentru date neliniare) are bias mare și varianță mică — fenomen numit *subînvățare* (eng. *underfitting*): nu reușește să surprindă structura datelor nici măcar pe setul de antrenament. Un model prea complex (de exemplu, un arbore foarte adânc) are bias mic, dar varianță mare — *supraînvățare* (eng. *overfitting*): se potrivește zgomotului din datele de antrenament și generalizează slab. Scopul practic al antrenării este găsirea unui echilibru între cele două surse de eroare [8].

Cele trei modele studiate acoperă, în mod intenționat, un spectru de complexitate: regresia logistică este un model liniar cu bias relativ mare și varianță mică; arborele de decizie individual este un model neliniar flexibil, cu bias mic, dar varianță mare; pădurea aleatoare reduce varianța arborelui prin mediere, păstrând bias-ul scăzut (Secțiunea 2.7). Mecanismele care controlează acest compromis se numesc, generic, *regularizare*: la regresia logistică, penalizarea L_2 a ponderilor; la arbore, limitarea adâncimii și a dimensiunii frunzelor; la pădure, agregarea mai multor arbori decorelați. Aceste mecanisme sunt detaliate în secțiunile dedicate fiecărui model.

2.4 Problema dezechilibrului de clase

Un set de date este *dezechilibrat* (eng. *imbalanced*) atunci când clasele nu sunt reprezentate în proporții apropiate. În cazul predicției podiumului, doar trei din aproximativ douăzeci de piloți ajung pe podium în fiecare cursă, deci clasa pozitivă reprezintă circa 15% din exemple — un raport de aproximativ 1:6 între clasa minoritară și cea majoritară. Dezechilibrul afectează atât *antrenarea* (funcția de cost este dominată de clasa majoritară, iar modelul tinde să o favorizeze), cât și *evaluarea* (metricile globale devin înșelătoare) [3].

La nivelul evaluării, problema este ilustrată de un clasificator trivial care prezice mereu clasa majoritară (“nu termină pe podium”): acesta obține o acuratețe de aproximativ 85% fără a avea nicio valoare practică, deoarece nu identifică niciodată un pilot de podium. Prin urmare, acuratețea globală nu este o măsură adecvată în prezența dezechilibrului, aspect reluat în analiza din Capitolul 4. Evaluarea trebuie

completată cu metrici care surprind performanța pe clasa minoritară — precizia, recall-ul și scorul F_1 — și cu metrici specifice problemei, calculate la nivel de cursă (Secțiunea 2.10). În literatura dedicată datelor dezechilibrate se subliniază, în plus, că graficul precizie–recall este mai informativ decât curba ROC atunci când clasa pozitivă este rară, fiindcă reflectă direct costul fals-pozitivelor relativ la puținele exemple pozitive [4], [11].

La nivelul antrenării, literatura propune trei familii de tehnici de atenuare a dezechilibrului [3]. Prima este *reeșantionarea* datelor: fie suprareprezentarea clasei minoritare (eng. *oversampling*), fie subreprezentarea clasei majoritare (eng. *undersampling*). O metodă consacrată din această familie este SMOTE (eng. *Synthetic Minority Over-sampling Technique*), care generează exemple sintetice prin interpolarea între exemple minoritare vecine, în loc să le dubleze pe cele existente [12]. A doua familie este *învățarea sensibilă la cost* (eng. *cost-sensitive learning*), în care erorile pe clasa minoritară primesc o pondere mai mare în funcția de cost. A treia constă în ajustarea *pragului de decizie*, deplasându-l față de valoarea implicită de 0,5 pentru a echilibra precizia și recall-ul. În această lucrare se adoptă abordarea sensibilă la cost: la regresia logistică, fiecare exemplu primește o pondere invers proporțională cu frecvența clasei sale (Secțiunea 2.5), astfel încât erorile pe clasa de podium să contribuie semnificativ la funcția de cost. Această alegere evită introducerea de exemple sintetice și păstrează structura cauzală a datelor (Secțiunea 2.8).

2.5 Regresia logistică

Regresia logistică este unul dintre cele mai folosite modele de clasificare, apreciat pentru simplitate, eficiență și interpretabilitate [7], [8]. În ciuda denumirii, este un model de *clasificare*, nu de regresie: modelează probabilitatea clasei pozitive printr-o transformare neliniară a unei combinații liniare de caracteristici.

2.5.1 Modelul și interpretarea log-odds

Fie $\mathbf{w} \in \mathbb{R}^d$ vectorul de ponderi și $b \in \mathbb{R}$ termenul liber. Modelul calculează un scor liniar $z = \mathbf{w}^\top \mathbf{x} + b$ și îl transformă într-o probabilitate prin funcția *logistică* (sau *sigmoid*):

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b). \quad (2.3)$$

Funcția sigmoid este monoton crescătoare, simetrică în jurul punctului $(0, 0,5)$ și comprimă orice valoare reală în intervalul $(0, 1)$, putând fi astfel interpretată ca probabilitate. Sensul ponderilor devine transparent dacă rescriem modelul în termeni de *cote logaritmice* (eng. *log-odds*). Inversând relația (2.3), se obține:

$$\log \frac{\hat{p}}{1 - \hat{p}} = \mathbf{w}^\top \mathbf{x} + b. \quad (2.4)$$

Cu alte cuvinte, regresia logistică presupune că logaritmul raportului dintre șansa de podium și șansa de non-podium este liniar în caracteristici. O creștere cu o unitate a caracteristicii j modifică log-odds-ul cu w_j , deci înmulțește cota cu e^{w_j} . Această proprietate face modelul ușor de interpretat: semnul și magnitudinea fiecărei ponderi indică direcția și tăria influenței caracteristicii respective.

2.5.2 Estimarea prin maximul verosimilității și entropia încrucișată

Ponderile se determină prin principiul *maximului verosimilității*. Presupunând exemple independente, verosimilitatea modelului este $\prod_{i=1}^N \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}$. Maximizarea logaritmului verosimilității este echivalentă cu minimizarea *entropiei încrucișate* (eng. *cross-entropy*) negate, la care se adaugă un termen de regularizare L_2 ce penalizează ponderile mari și combate supraînvățarea (Secțiunea 2.3):

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (2.5)$$

Un avantaj teoretic important este că această funcție de cost este *convexă* în raport cu parametrii [13]: nu există minime locale, ci un singur minim global, ceea ce garantează că o metodă de optimizare bine reglată converge către soluția optimă. Alegerea entropiei încrucișate în locul erorii pătratice este esențială: combinată cu sigmoidul, ea conduce la un gradient simplu și evită saturarea derivatei care ar încetini învățarea.

2.5.3 Coborârea pe gradient

Spre deosebire de regresia liniară, ecuația $\nabla \mathcal{L} = 0$ nu are soluție în formă închisă, astfel încât minimizarea se realizează numeric, prin *coborâre pe gradient* (eng. *gradient descent*) [13]. Ponderile sunt inițializate (aici cu zero) și actualizate iterativ în sensul opus gradientului, cu un pas de învățare η : $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$. Gradientul funcției de cost față de ponderi are forma compactă, în care $\mathbf{X}$ este matricea exemplilor, $\hat{\mathbf{p}}$ vectorul probabilităților prezise și $\mathbf{y}$ vectorul etichetelor:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}) + \lambda \mathbf{w}. \quad (2.6)$$

Termenul $(\hat{\mathbf{p}} - \mathbf{y})$ este *reziduul* predicției; intuitiv, contribuția fiecărui exemplu la actualizare este proporțională cu cât de mult “greșește” modelul pe el. În funcție de câte exemple se folosesc la fiecare actualizare, se disting trei variante [7]: varianta *batch* (gradientul se calculează pe întreg setul, ca în relația (2.6)), varianta *stocastică* (un singur exemplu pe pas) și varianta *mini-batch* (un subset). Varianta batch oferă actualizări stabile și a fost adoptată în această lucrare, dat fiind volumul moderat al setului de date. Pasul de învățare η controlează viteza convergenței: o valoare prea mare poate provoca oscilații sau divergență, iar una prea mică încetinește antrenarea. Convergența poate fi monitorizată urmărind evoluția funcției de cost de la o iterație la alta.

2.5.4 Regularizarea și standardizarea

Termenul $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ din relația (2.5) reprezintă *regularizarea* L_2 (sau *ridge*), care descurajează ponderile mari și, implicit, modelele excesiv de încrezătoare; coeficientul λ reglează intensitatea penalizării [7]. O alternativă este regularizarea L_1 (*lasso*), care penalizează suma valorilor absolute ale ponderilor și are proprietatea de a anula complet unele ponderi, realizând astfel o selecție de caracteristici. În această lucrare se folosește regularizarea L_2 , mai stabilă numeric pentru optimizarea pe gradient.

Deoarece coborârea pe gradient și penalizarea L_2 sunt sensibile la scara caracteristicilor, o etapă de preprocesare necesară este *standardizarea* (eng. *z-score*): fiecare caracteristică este transformată să aibă medie zero și abatere standard unitară, $x' = (x - \mu)/\sigma$. În absența standardizării, caracteristicile cu valori numerice mari ar domina gradientul și ar fi penalizate disproporționat. Esențial, parametrii standardizării (μ și σ) se estimează *exclusiv* pe datele de antrenament și se reaplică identic la validare și test, pentru a nu introduce scurgere de informație (Secțiunea 2.8).

Pentru a contracara dezechilibrul de clase (Secțiunea 2.4), fiecare exemplu poate primi în plus o pondere invers proporțională cu frecvența clasei sale, astfel încât erorile pe clasa minoritară (podium) să contribuie semnificativ la funcția de cost.

2.6 Arbori de decizie

Arborii de decizie sunt modele neparametrice care partiționează recursiv spațiul caracteristicilor printr-o succesiune de teste simple, organizate sub forma unei structuri arborescente. Familia a fost dezvoltată în paralel în statistică — prin algoritmul CART (eng. *Classification and Regression Trees*) [14] — și în inteligența artificială, prin algoritmi ID3 și C4.5 ai lui Quinlan [15], [16]. Implementarea din această lucrare urmează modelul CART pentru clasificare binară.

2.6.1 Partiționarea recursivă

Un arbore de decizie este alcătuit din *noduri interne*, fiecare aplicând un test de forma “caracteristica j este $\leq$ pragul t ?”, și din *frunze*, care stochează o predicție. Pornind de la rădăcină, fiecare exemplu este rutat spre stânga sau spre dreapta în funcție de rezultatul testului, până ajunge într-o frunză. În cazul de față, frunza stochează fracția de exemple pozitive din ea, interpretată drept probabilitate de podium. Figura 2.1 ilustrează schematic această structură.

Construirea arborelui este *lacomă* (eng. *greedy*) și de sus în jos: la fiecare nod se alege divizarea care optimizează local un criteriu, fără a reconsidera deciziile anterioare. Această strategie nu garantează arborele global optim (problemă NP-dificilă), dar este eficientă și produce, în practică, arbori buni [7].

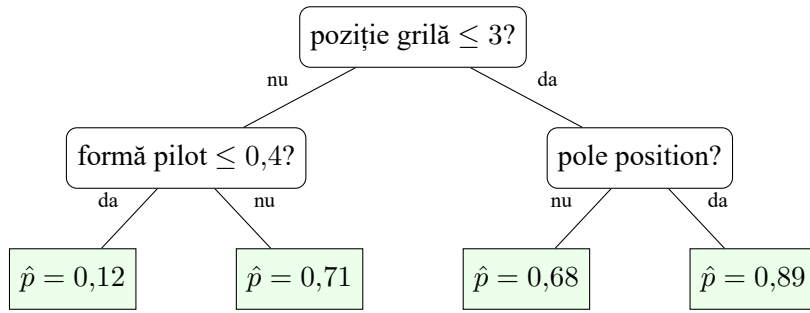


Figura 2.1 Exemplu schematic de arbore de decizie pentru predicția podiumului. Fiecare nod intern aplică un test asupra unei caracteristici, iar frunzele estimează probabilitatea de podium $\hat{p}$.

2.6.2 Măsuri de impuritate

Calitatea unei divizări se evaluează prin reducerea *impurității* nodurilor. O măsură de impuritate este maximă atunci când clasele sunt amestecate în proporții egale și nulă atunci când nodul conține o singură clasă (nod *pur*). Criteriul folosit în această lucrare este *câștigul de informație* (eng. *information gain*), bazat pe *entropie*. Pentru un nod cu proporția p de exemple pozitive, entropia binară este:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p), \quad (2.7)$$

cu convenția $0 \log_2 0 = 0$. Câștigul de informație al unei divizări care împarte un nod părinte în doi copii (stânga și dreapta) este reducerea de impuritate:

$$IG = H(\text{părinte}) - \frac{n_{st}}{n} H(\text{stânga}) - \frac{n_{dr}}{n} H(\text{dreapta}). \quad (2.8)$$

Algoritmul alege, la fiecare nod, perechea (caracteristică, prag) care maximizează câștigul de informație.

Entropia nu este singura măsură de impuritate. Tabelul 2.1 rezumă cele trei măsuri uzuale pentru clasificarea binară, exprimate în funcție de proporția p a clasei pozitive [7], [14]. Indicele Gini, folosit implicit de CART, este apropiat numeric de entropie, dar evită calculul logaritmului; eroarea de clasificare este prea grosieră pentru a ghida creșterea arborelui și se folosește mai degrabă la retezare. În practică, alegerea între Gini și entropie are un efect minor asupra performanței.

Tabelul 2.1 Măsuri uzuale de impuritate pentru clasificarea binară (proporția clasei pozitive p).

Măsură de impuritate	Formulă	Valoare maximă ($p = 0,5$)
Entropie (biți)	$-p \log_2 p - (1 - p) \log_2 (1 - p)$	1,00
Indice Gini	$2p(1 - p)$	0,50
Eroare de clasificare	$1 - \max(p, 1 - p)$	0,50

2.6.3 Criterii de oprire, supraînvăţare şi instabilitate

Lăsat să crească nelimitat, un arbore poate izola fiecare exemplu de antrenament într-o frunză proprie, atingând impuritate nulă, dar supraînvăţând masiv (Secţiunea 2.3). Creşterea se controlează, de aceea, prin criterii de oprire (eng. *pre-pruning*): adâncimea maximă (`max_depth`), numărul minim de exemple necesare pentru o divizare (`min_samples_split`) şi numărul minim de exemple într-o frunză (`min_samples_leaf`). O alternativă este *retezarea* ulterioară (eng. *post-pruning*), în care arborele este crescut complet şi apoi simplificat eliminând ramurile care nu îmbunătăţesc performanţa pe un set de validare [14].

Arborii de decizie au avantaje notabile: sunt uşor de interpretat (o predicţie corespunde unui lanţ de reguli “dacă—atunci”), surprind interacţiuni neliniare între caracteristici şi nu necesită standardizarea datelor, fiind invarianţi la transformări monotone ale caracteristicilor [8]. Dezavantajul lor major este *instabilitatea*: o mică perturbare a datelor de antrenament poate schimba radical structura arborelui, deci arborii individuali au varianţă mare. Tocmai această slăbiciune motivează metodele de ansamblu din secţiunea următoare, care reduc varianţa prin agregarea mai multor arbori.

2.7 Pădurea aleatoare (Random Forest)

2.7.1 Justificarea teoretică a ansamblurilor

Metodele de *ansamblu* (eng. *ensemble methods*) combină predicţiile mai multor modele de bază pentru a obţine un model agregat mai performant decât oricare model individual [17]. Intuiţia provine din descompunerea bias–varianţă (Secţiunea 2.3): media mai multor modele cu bias mic, dar varianţă mare reduce varianţa fără a creşte bias-ul. Concret, dacă se mediază T predictorii, fiecare cu varianţa σ^2 şi corelaţie medie ρ între ei, varianţa mediei este:

$$\text{Var}\left(\frac{1}{T} \sum_{t=1}^T \hat{f}_t\right) = \rho \sigma^2 + \frac{1-\rho}{T} \sigma^2. \quad (2.9)$$

Relaţia (2.9) arată două lucruri esenţiale [7]: pe măsură ce numărul de modele T creşte, al doilea termen tinde la zero, dar primul termen rămâne mărginit de corelaţia ρ dintre modele. Prin urmare, beneficiul ansamblului depinde decisiv de *decorelarea* modelelor de bază — a face modelele cât mai diferite între ele este la fel de important ca a le face individuale bune.

2.7.2 Bagging şi pădurea aleatoare

Pădurea aleatoare (eng. *Random Forest*) este o metodă de ansamblu introdusă de Breiman [18], care combină arbori de decizie folosind doi factori de aleator pentru a-i decorela (a reduce ρ din rela-

ția (2.9)):

- **Bagging** (eng. *bootstrap aggregating*): fiecare arbore este antrenat pe un eșantion *bootstrap*, obținut prin extragerea cu revenire a n exemple din setul de antrenament de dimensiune n . Astfel, fiecare arbore vede o variantă ușor diferită a datelor.
- **Subspații aleatoare de caracteristici** (eng. *random subspace*): la fiecare divizare se consideră doar un subset aleator de caracteristici (uzual $\sqrt{d}$ din cele d disponibile), nu toate. Această alegere împiedică toți arborii să folosească aceleași câteva caracteristici dominante, decorelându-i suplimentar.

Predicția finală este media probabilităților produse de toți arborii. Figura 2.2 ilustrează schematic acest flux. Datorită medierii, pădurea aleatoare păstrează bias-ul mic al arborilor individuali, dar reduce substanțial varianța lor, fiind în general robustă, puțin sensibilă la hiperparametri și competitivă pe date tabelare [7], [18].

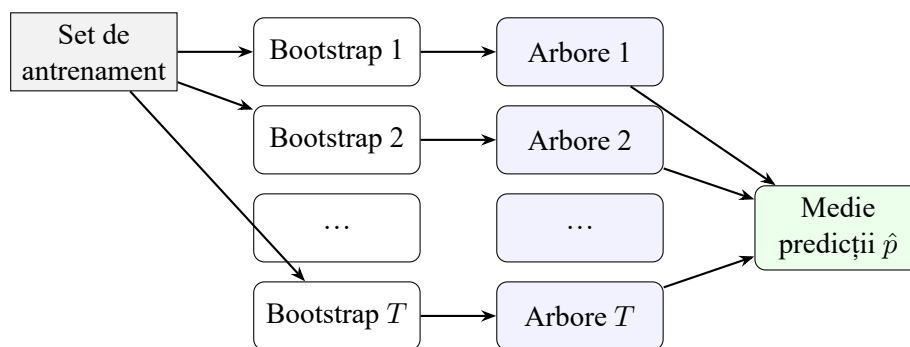


Figura 2.2 Schema pădurii aleatoare: din setul de antrenament se extrag T eșantioane bootstrap, pe fiecare se crește un arbore cu subset aleator de caracteristici, iar predicțiile se mediază.

2.7.3 Estimarea out-of-bag și importanța caracteristicilor

Un avantaj practic al bagging-ului este estimarea *out-of-bag* (OOB). Deoarece un eșantion bootstrap omite, în medie, aproximativ 37% dintre exemple ($\lim_{n \rightarrow \infty} (1 - 1/n)^n = e^{-1}$), fiecare exemplu este “în afara sacului” pentru o parte dintre arbori. Predicția OOB a unui exemplu se calculează folosind doar arborii care nu l-au inclus în eșantionul lor, oferind astfel o estimare a erorii de generalizare *fără* a fi nevoie de un set de validare separat [18].

Pădurea aleatoare oferă și o măsură de *importanță a caracteristicilor*. Cea folosită aici este reducerea medie de impuritate (eng. *mean decrease in impurity*, MDI): pentru fiecare caracteristică se însumează câștigurile de informație ale divizărilor care o folosesc, ponderate cu numărul de exemple din nod, mediate apoi peste toți arborii și normalizate. Această măsură adaugă un grad de interpretabilitate unui model altfel de tip “cutie neagră” și este analizată în Capitolul 4.

2.7.4 Alte metode de ansamblu

Pe lângă bagging, a doua mare familie de ansambluri este *boosting*-ul, care antrenează modelele secvențial, fiecare model nou concentrându-se asupra exemplelor greșite de cele anterioare. Algoritmul AdaBoost [19] și, ulterior, *gradient boosting* [20] — cu implementări eficiente precum XGBoost [21] — sunt printre cele mai performante metode pe date tabelare. Spre deosebire de bagging, care reduce în principal varianța, boosting-ul reduce și bias-ul, dar este mai sensibil la zgomot și la reglarea hiperparametrilor. În această lucrare s-a ales pădurea aleatoare ca metodă de ansamblu reprezentativă, lăsând boosting-ul ca direcție de dezvoltare ulterioară (Capitolul 5).

2.8 Ingineria caracteristicilor și scurgerea de informație

Calitatea unui model de învățare automată depinde decisiv de caracteristicile de intrare — adesea mai mult decât de alegerea algoritmului [8]. *Ingineria caracteristicilor* (eng. *feature engineering*) este procesul de transformare a datelor brute în variabile care expun cât mai clar structura relevantă pentru predicție. Pentru predicția podiumului, datele brute (poziții, puncte, timpi) sunt agregate în indicatori de formă, de forță a echipei și de istoric pe circuit, descriși în Capitolul 3.

În problemele cu o componentă temporală, precum predicția unei curse viitoare, apare un risc metodologic major: *scurgerea de informație* (eng. *data leakage*). Aceasta înseamnă utilizarea, la antrenare sau la calculul caracteristicilor, a unor informații care nu ar fi disponibile în momentul predicției reale. Scurgerea conduce la o supraestimare optimistă a performanței în experiment, urmată de o prăbușire a acesteia în exploatarea reală. Se disting mai multe forme de scurgere relevante pentru această lucrare:

- **Scurgerea țintei** (eng. *target leakage*): o caracteristică încorporează, direct sau indirect, eticheta pe care încercăm să o prezicem. De exemplu, folosirea numărului total de puncte acumulate de un pilot *într-un sezon complet* ar include punctele din cursa pe care vrem să o prezicem.
- **Scurgerea temporală** (eng. *look-ahead bias*): folosirea unor date din viitorul cursei prezise, chiar dacă nu provin direct din etichetă (de pildă, media de sosire calculată pe întregul sezon, inclusiv curse viitoare).
- **Scurgerea prin preprocesare**: estimarea unor parametri de transformare (precum media și abaterea standard pentru standardizare) pe întregul set de date, inclusiv pe cel de test, în loc de a-i estima doar pe antrenament (Secțiunea 2.5).

Pentru a evita aceste fenomene, toate caracteristicile trebuie să fie *strict cauzale*, adică să folosească exclusiv informații anterioare cursei prezise. Strategia adoptată constă în procesarea curselor în ordine cronologică și menținerea unei “stări” istorice (sume și medii rulante), din care se calculează caracteristicile cursei curente *înainte* de a fi actualizată cu rezultatul acelei curse. Acest principiu cauzal,

aplicat consecvent, este condiția necesară pentru ca estimarea performanței din Secțiunea 2.9 să fie corectă.

2.9 Validarea și estimarea performanței

Estimarea corectă a performanței necesită evaluarea modelului pe date care nu au fost folosite la antrenare [22]. Cea mai simplă schemă este împărțirea datelor în trei seturi disjuncte: un set de *antrenament*, pe care se învață parametrii modelului; un set de *validare*, pe care se aleg hiperparametrii și se compară variante de model; și un set de *test*, folosit o singură dată, la final, pentru a raporta o estimare nepărtinitoare a performanței [7]. Separarea validării de test este esențială: dacă hiperparametrii ar fi reglați direct pe setul de test, estimarea finală ar deveni optimistă.

Pentru a folosi mai eficient datele limitate, se utilizează frecvent *validarea încrucișată k -fold*: setul este împărțit în k părți, modelul fiind antrenat pe $k - 1$ și evaluat pe partea rămasă, rotativ, performanța raportată fiind media celor k estimări [22]. Această procedură presupune însă că exemplele sunt *independente și interschimbabile* (eng. *i.i.d.*), ipoteză care **nu** este valabilă pentru date temporale. A antrena pe curse viitoare pentru a prezice curse trecute ar reprezenta o formă de scurgere temporală (Secțiunea 2.8) și ar supraestima performanța, întrucât amestecarea aleatoare a exemplurilor în pliuri rupe ordinea cronologică [23].

Soluția consacrată pentru serii temporale este *validarea cu origine glisantă* (eng. *rolling-origin* sau *walk-forward validation*), în care modelul este întotdeauna antrenat pe trecut și evaluat pe viitor, originea de separare avansând în timp [23]. În această lucrare se folosește o variantă pe ani, cu separare strictă în trei seturi: pentru fiecare sezon de test S , modelul este antrenat pe toate sezoanele până la $S - 2$ inclusiv (adică $< S - 1$), sezonul $S - 1$ este ținut deoparte ca set de *validare*, iar performanța finală se raportează pe sezonul nevăzut S (setul de *test*). Astfel se prezice mereu “viitorul” cu modele antrenate doar pe “trecut”, iar sezonul de validare rămâne complet separat de cel de antrenare, eliminând scurgerea de informație temporală. Granularitatea pe sezon (și nu pe cursă) este firească pentru Formula 1, deoarece regulamentul tehnic și componența echipelor sunt stabile în interiorul unui sezon, dar se pot modifica semnificativ între sezoane.

2.10 Metrici de evaluare

Evaluarea unui clasificator binar pornește de la *matricea de confuzie* (Tabelul 2.2), care încrucișează eticheta reală cu cea prezisă în patru categorii: adevărat pozitive (TP), fals pozitive (FP), adevărat negative (TN) și fals negative (FN) [11]. Pe baza acestor patru numere se construiesc toate metricile de clasificare folosite în lucrare.

Tabelul 2.2 Matricea de confuzie pentru clasificarea binară a etichetei de podium.

	Prezis: podium	Prezis: non-podium
Real: podium	Adevărat pozitiv (TP)	Fals negativ (FN)
Real: non-podium	Fals pozitiv (FP)	Adevărat negativ (TN)

Plecând de la aceste mărimi, se definesc:

$$\text{Acuratețe} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precizie} = \frac{TP}{TP + FP}, \quad (2.10)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precizie} \cdot \text{Recall}}{\text{Precizie} + \text{Recall}}. \quad (2.11)$$

Precizia răspunde la întrebarea “dintre piloții preziși pe podium, câți chiar au ajuns acolo?”, iar *recall-ul* la “dintre piloții care au ajuns pe podium, câți au fost identificați?”. Cele două sunt în tensiune, iar scorul F_1 — media lor armonică — le echilibrează într-o singură valoare. În prezența dezechilibrului de clase (Secțiunea 2.4), F_1 este preferat acurateței, deoarece nu este dominat de clasa majoritară.

O modalitate de evaluare independentă de pragul de decizie este *curba ROC* (eng. *Receiver Operating Characteristic*), care reprezintă rata de adevărat pozitive în funcție de rata de fals pozitive, pe măsură ce pragul variază; aria de sub curbă (AUC) rezumă această performanță într-un singur număr [11]. Totuși, atunci când clasa pozitivă este rară, curba precizie–recall este mai informativă decât curba ROC, fiindcă reflectă mai fidel costul fals-pozitivelor (Secțiunea 2.4) [4].

Pe lângă metricile generale de clasificare, soluția definește metrici *specifice problemei*, calculate la nivel de cursă și apoi mediate pe sezon. Acestea sunt necesare deoarece obiectivul real nu este eticheta individuală a fiecărui pilot, ci podiumul corect al fiecărei curse. Pentru fiecare cursă, podiumul prezis (primii trei piloți după probabilitatea de podium) este comparat cu podiumul real (primii trei după poziția finală):

- *rata de nimerire a podiumului* (eng. *podium hit rate*) — fracția medie a celor trei piloți preziși care se regăsesc în podiumul real;
- *podiumul exact* (eng. *exact podium*) — fracția curselor în care mulțimea celor trei piloți preziși coincide exact cu cea reală (indiferent de ordine);
- *acuratețea câștigătorului* (Top-1) — fracția curselor în care pilotul prezis pe prima poziție este cel care câștigă efectiv cursa.

Aceste metrici corespund evaluării de tip “top- k ” din regăsirea informației și ordonare (Secțiunea 2.2) [9]. Împreună cu acuratețea clasificării binare, ele stau la baza studiului comparativ din Capitolul 4.

2.11 Calibrarea probabilităților

Deoarece soluția folosește probabilitățile $\hat{p}$ atât pentru ordonarea piloților, cât și pentru agregarea într-un clasament de campionat estimat, contează nu doar *ordinea* indusă de probabilități, ci și *valoarea* lor absolută. Un clasificator este *calibrat* dacă, dintre exemplele cărora le atribuie o probabilitate $\hat{p}$, o fracție de aproximativ $\hat{p}$ sunt efectiv pozitive [24]. Calibrarea nu coincide cu puterea discriminativă: un model poate ordona corect exemplele, dar produce probabilități sistematic prea mari sau prea mici.

Niculescu-Mizil și Caruana [24] arată că diferite familii de modele au comportamente de calibrare distincte: regresia logistică, antrenată direct prin minimizarea entropiei încrucișate, produce în mod natural probabilități bine calibrate, în timp ce metodele de ansamblu bazate pe arbori pot prezenta abateri sistematice. Atunci când calibrarea este importantă, probabilitățile pot fi corectate ulterior prin tehnici precum scalarea Platt (o regresie logistică aplicată scorurilor) sau regresia izotonică. În această lucrare, probabilitățile sunt folosite predominant ca scoruri de ordonare, ceea ce reduce sensibilitatea la o calibrare imperfectă; calibrarea explicită rămâne o direcție de îmbunătățire (Capitolul 5).

2.12 Sinteză comparativă a algoritmilor studiați

Cele trei modele alese acoperă, intenționat, un spectru de complexitate și de compromisuri bias–varianță (Secțiunea 2.3). Tabelul 2.3 sintetizează proprietățile lor teoretice relevante pentru studiul comparativ din Capitolul 4.

Tabelul 2.3 Sinteză comparativă a proprietăților teoretice ale celor trei algoritmi.

Proprietate	Regresie logistică	Arbore de decizie	Pădure aleatoare
Tip graniță	liniară	neliniară	neliniară
Bias / varianță	bias mare / var. mică	bias mic / var. mare	bias mic / var. redusă
Probabilitate	sigmoid (calibrată)	fracție în frunză	medie pe arbori
Interpretabilitate	ridicată (ponderi)	ridicată (reguli)	medie (importante)
Standardizare	necesară	nu	nu
Regularizare	L_2 pe ponderi	oprire / retezare	mediere (ansamblu)

2.13 Lucrări conexe

Aplicarea învățării automate la predicția rezultatelor sportive a fost studiată pe larg. Bunker și Thabtah [25] propun un cadru general (eng. *framework*) pentru predicția rezultatelor sportive, în care subliniază importanța ingineriei caracteristicilor și a unei evaluări care respectă ordinea cronologică a meciurilor — exact principiile aplicate și în această lucrare. Autorii observă că metodele bazate pe

arbori și rețelele neuronale sunt cele mai frecvent folosite, iar acuratețea simplă este adesea o metrică insuficientă.

În privința algoritmilor, metodele de tip ansamblu — în special pădurile aleatoare — sunt raportate constant ca robuste și competitive pe date tabelare [7], [18], iar metodele de tip *gradient boosting* domină frecvent competițiile pe astfel de date [20], [21]. O abordare alternativă pentru clasificarea pe date cu multe caracteristici o constituie mașinile cu vectori suport (eng. *Support Vector Machines*) [26], care caută hiperplanul cu marginea maximă între clase; acestea nu produc însă, în mod nativ, probabilități, ceea ce le face mai puțin directe pentru reformularea de tip ordonare folosită aici.

Din punct de vedere metodologic, problema scurgerii de informație și a validării temporale a primit o atenție crescândă: Bergmeir și Benítez [23] demonstrează empiric de ce validarea încrucișată clasică este inadecvată pentru serii temporale și recomandă scheme de tip walk-forward, iar literatura privind clasele dezechilibrate [3], [4], [12] fundamentează alegerea metricilor și a tehnicilor de atenuare a dezechilibrului.

Față de aceste lucrări, contribuția de față se diferențiază prin trei aspecte: (i) implementarea *integrală*, de la zero, a celor trei algoritmi, ceea ce permite controlul complet și înțelegerea în profunzime a mecanismelor lor; (ii) o evaluare temporală riguroasă, cu separare în antrenare, validare și test pe ani, care elimină scurgerea de informație; și (iii) folosirea unor metrici specifice problemei, calculate la nivel de cursă, alături de o analiză a efectului datelor disponibile in-sezon (Capitolul 4).

Capitolul 3

Rezolvarea temei de proiect

Acest capitol descrie, în ordine logică, etapele parcurse în rezolvarea temei și justifică fiecare decizie de proiectare. Firul expunerii urmează drumul datelor prin sistem: pornind de la arhitectura de ansamblu (Secțiunea 3.1) și de la tehnologiile alese (Secțiunea 3.2), se trece prin sursa de date (Secțiunea 3.3), construirea tabelului de rezultate (Secțiunea 3.4) și ingineria caracteristicilor cauzale (Secțiunea 3.6), apoi prin implementarea celor trei modele (Secțiunea 3.7), transformarea probabilităților în predicții de podium și clasament (Secțiunea 3.8), procedura de validare temporală (Secțiunea 3.9), analiza antrenării incrementale in-sezon (Secțiunea 3.10), metricile și instrumentația experimentală (Secțiunea 3.11) și, în final, interfața în linie de comandă care leagă toate componentele (Secțiunea 3.12). Pentru fiecare etapă se explică nu doar *ce* a fost implementat, ci și *de ce* — în special raportat la cele două cerințe metodologice centrale ale lucrării: implementarea de la zero a algoritmilor și eliminarea completă a scurgerii de informație (Secțiunea 2.8).

3.1 Arhitectura sistemului

Sistemul este organizat ca o succesiune de etape (eng. *pipeline*), de la datele brute până la predicții și evaluare. Această structură liniară a fost aleasă deoarece reflectă fidel dependențele dintre etape: fiecare etapă consumă ieșirea celei precedente și produce o intrare clar definită pentru următoarea. Figura 3.1 prezintă schema de arhitectură: datele sunt încărcate din memoria cache locală, transformate într-un tabel de rezultate, apoi îmbogățite cu caracteristici cauzale; pe acest set se antrenează cele trei modele, ale căror probabilități sunt folosite atât pentru predicția podiumului per cursă, cât și pentru agregarea în clasament și pentru modulul de evaluare și vizualizare.

Separarea pe etape distincte aduce trei avantaje practice. În primul rând, fiecare etapă poate fi testată și validată independent — de exemplu, corectitudinea cauzală a caracteristicilor poate fi verificată izolat, fără a antrena vreun model. În al doilea rând, etapele costisitoare (încărcarea datelor și ingineria caracteristicilor) sunt executate o singură dată, iar rezultatul lor este memorat pe disc (Secțiunea 3.14),

astfel încât experimentele repetate cu modele diferite refolosesc același set de caracteristici. În al treilea rând, modularitatea permite înlocuirea unei componente (de pildă, adăugarea unui al patrulea model) fără a modifica restul sistemului.

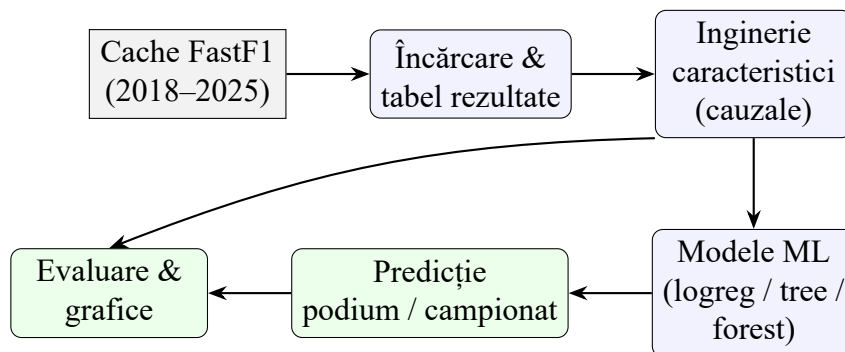


Figura 3.1 Arhitectura sistemului de predicție: de la datele brute la evaluare.

3.2 Tehnologii folosite și organizarea codului

Implementarea este realizată în limbajul Python, ales pentru ecosistemul său bogat de biblioteci de calcul numeric și pentru lizibilitatea care facilitează expunerea algoritmilor. Codul este organizat ca un pachet, `f1pred`, structurat pe subpachete corespunzătoare etapelor din Figura 3.1; Tabelul 3.1 rezumă această organizare. Separarea pe module urmărește principiul responsabilității unice: fiecare modul rezolvă o singură etapă și expune o interfață restrânsă către celelalte.

Tabelul 3.1 Organizarea pe module a pachetului `f1pred`.

Modul	Responsabilitate
<code>data/loader</code>	acces de nivel jos la cache-ul FastF1 (sesiuni R și Q, vreme)
<code>data/build_dataset</code>	asamblarea tabelului “lung” de rezultate
<code>features/engineering</code>	ingineria caracteristicilor cauzale și eticheta
<code>models/</code>	cele trei modele (<code>logreg</code> , <code>tree</code> , <code>forest</code>)
<code>predict/</code>	podium per cursă și clasament de campionat
<code>evaluation/</code>	metrici și validarea temporală
<code>viz/plots</code>	generarea graficelor
<code>config</code>	căi, sezoane, hiperparametri, sistemul de puncte
<code>cli</code>	interfața în linie de comandă

Bibliotecile externe sunt folosite strict pentru sarcini auxiliare, niciuna nefiind o bibliotecă de învățare automată: NumPy [1] pentru algebra vectorizată (matrice, sume cumulative, operații pe vectori), pandas [27] pentru manipularea datelor tabelare și gruparea pe curse, matplotlib [28] pentru grafice și FastF1 [29] pentru accesul la datele Formula 1. Această delimitare este o cerință metodologică explicită a temei: toți algoritmii de învățare, criteriile de divizare, metricile și procedura de antrenare sunt implementați manual, peste NumPy. Renunțarea la o bibliotecă consacrată precum scikit-learn [2] are un cost — mai mult cod de scris și de testat — dar oferă două câștiguri esențiale pentru o lucrare

didactică: controlul complet asupra fiecărui detaliu (de la inițializarea ponderilor până la departajarea egalităților) și transparența mecanismelor, care altfel ar rămâne ascunse în spatele apelurilor de bibliotecă.

3.3 Sursa de date

Datele provin din interfața de programare FastF1 [29], o bibliotecă Python care oferă acces la rezultatele oficiale, la programul competițional și la informațiile de cronometraj din Formula 1, agregând surse precum API-ul Ergast/Jolpica și fluxul de date al competiției. Au fost folosite opt sezoane consecutive, 2018–2025, însumând 3458 de rezultate (perechi cursă–pilot) provenite de la 43 de piloți și 19 echipe. Intervalul a fost ales pentru a fi suficient de lung încât să permită validarea temporală pe mai multe sezoane (Secțiunea 3.9) și, totodată, suficient de recent încât regulamentul tehnic și sistemul de punctare să fie relativ omogene.

Pentru fiecare rundă se încarcă două sesiuni: cursa propriu-zisă (codul R), din care se extrag poziția pe grilă, poziția finală, statusul de clasare, punctele și echipa fiecărui pilot, și sesiunea de calificări (codul Q), din care se preia poziția în calificări. La acestea se adaugă un rezumat al condițiilor meteo (prezența ploii, temperatura pistei și a aerului), calculat din datele meteorologice ale sesiunii. Sesiunile indisponibile în cache (de exemplu, curse anulate sau sesiuni nedescărcate) sunt sărite cu un avertisment, fără a întrerupe procesul.

O decizie de proiectare importantă este accesul *strict offline*: FastF1 este configurat să citească exclusiv dintr-o memorie cache locală, fără nicio cerere de rețea. Această alegere are o dublă justificare. Pe de o parte, asigură *reproductibilitatea*: rezultatele experimentelor nu depind de momentul rulării sau de o eventuală modificare a datelor la sursă. Pe de altă parte, elimină dependența de conexiunea la internet și evită limitarea de rată (eng. *rate limiting*) impusă de API-ul public, care altfel ar întrerupe încărcarea unui volum mare de sesiuni.

3.4 Construirea setului de date

Prima etapă transformă datele brute, împrăștiate pe sesiuni, într-un singur tabel “lung” (eng. *long format*), cu o linie pentru fiecare pereche (sezon, rundă, pilot). Formatul lung a fost preferat unuia “lat” (o coloană per pilot) deoarece numărul de piloți variază de la sezon la sezon, iar reprezentarea pe linii independente se potrivește natural atât antrenării modelelor (fiecare linie este un exemplu), cât și gardării pe curse necesare la evaluare. Tabelul conține coloanele rezumate în Tabelul 3.2.

Două aspecte merită justificate. Primul este tratarea *abandonurilor* (eng. *DNF — Did Not Finish*): din câmpul textual de status oferit de FastF1 se derivă un indicator binar de finalizare, considerând clasat orice pilot al cărui status este “Finished” sau de forma “+1 tur” etc.; acest indicator alimentează ulterior

Tabelul 3.2 Coloanele tabelului “lung” de rezultate (intrarea pentru ingineria caracteristicilor).

Coloană	Descriere
season, round	sezonul și runda (identifică unic o cursă)
driver, team	codul pilotului și numele echipei
grid, quali_pos	poziția pe grilă și poziția în calificări
position	poziția finală în cursă
status, finished	statusul de clasare și indicatorul de finalizare
points	punctele obținute în cursă
is_sprint_weekend	1 dacă weekendul a inclus o cursă de sprint
rain, track_temp, air_temp	rezumatul condițiilor meteo
event_name, circuit, event_date	contextul evenimentului

caracteristica de rată a abandonurilor (Secțiunea 3.6). Al doilea este *sortarea cronologică* stabilă, după (sezon, rundă, poziție): aceasta nu este o simplă convenție de afișare, ci o precondiție obligatorie pentru calculul corect al caracteristicilor rulante — procesarea curselor în ordine garantează că starea istorică din care se derivă o caracteristică conține exclusiv trecutul (Secțiunea 3.6). În fine, tabelul rezultat este salvat în format Parquet, un format binar coloană cu compresie, ales pentru viteza de citire și pentru păstrarea tipurilor de date, astfel încât etapele ulterioare să îl poată reutiliza fără a relua încărcarea din cache.

3.5 Analiza exploratorie a setului de date

Înainte de ingineria caracteristicilor, o scurtă analiză exploratorie justifică alegerile metodologice ulterioare. Tabelul 3.3 prezintă numărul de curse și de rezultate pe fiecare sezon. Se remarcă două aspecte. Primul este variabilitatea numărului de curse: de la 17 în 2020 (sezon scurtat din cauza pandemiei) până la 24 în 2024 și 2025; această neuniformitate confirmă oportunitatea formatului “lung”, care nu impune un număr fix de curse pe sezon. Al doilea este consecvența ratei de podium în jurul valorii de 0,15, dictată aritmetic de raportul dintre cele trei locuri de podium și cei aproximativ douăzeci de piloți din fiecare cursă.

Tabelul 3.3 Distribuția curselor și a rezultatelor pe sezoane.

Sezon	2018	2019	2020	2021	2022	2023	2024	2025	Total
Curse	21	21	17	22	22	22	24	24	173
Rezultate									3458
									(perechi cursă–pilot)

Pe întregul set, eticheta de podium are media 0,150, ceea ce înseamnă că un clasificator trivial care nu prezice niciodată un podium ar obține o acuratețe de circa 85% — reper care fundamentează discuția privind dezechilibrul de clase (Secțiunea 2.4) și alegerea metricilor specifice problemei (Secțiunea 2.10). Tot această analiză confirmă rolul dominant al poziției de start: marea majoritate a piloților

care urcă pe podium pleacă din primele rânduri ale grilei, observație care motivează includerea poziției pe grilă și a indicatorilor derivați printre caracteristicile cele mai importante (Secțiunea 3.6).

3.6 Ingineria caracteristicilor cauzale

Aceasta este etapa centrală a întregii soluții, deoarece de calitatea și corectitudinea caracteristicilor depinde atât performanța modelelor, cât și validitatea metodologică a întregului studiu. Pe baza considerentelor din Secțiunea 2.8, au fost construite 29 de caracteristici numerice, toate *strict cauzale*. Acestea se grupează în șase categorii, rezumate în Tabelul 3.4: poziția pe grilă și în calificări (cunoscute înainte de cursă), forma rulantă a pilotului (medii pe ultimele cinci curse), statistici cumulate la nivel de sezon, forța echipei, istoricul pe circuitul respectiv și contextul cursei (experiență, condiții meteo).

Tabelul 3.4 Categoriile de caracteristici folosite de modele.

Categorie	Exemple de caracteristici	Nr.
Grilă / calificări	poziție grilă, poziție calificări, pole	5
Formă rulantă pilot	medie sosire, medie puncte, rată podium, rată abandon	6
Sezon-to-date pilot	puncte, poziție, podiumuri, victorii, curse	5
Forța echipei	formă echipă, puncte echipă, poziție echipă	4
Istoric circuit	medie sosire pilot / echipă, rată podium pe circuit	3
Context	experiență, rundă, weekend cu sprint, vreme	6
Total		29

Fiecare categorie de caracteristici a fost concepută pentru a surprinde un factor diferit care influențează rezultatul unei curse:

- **Grilă și calificări** — poziția de start este, intuitiv, cel mai puternic predictor al rezultatului, fiind cunoscută cu certitudine înainte de cursă; pe lângă valorile brute, se adaugă indicatori binari pentru pole position și pentru plecarea din primele trei, respectiv primele zece poziții.
- **Forma rulantă a pilotului** — medii pe ultimele cinci curse pentru poziția de sosire, puncte, rata de podium, rata de abandon și poziția pe grilă, plus ultimul rezultat; aceste mărimi captează “momentul de formă” recent al pilotului.
- **Statistici sezon-to-date** — puncte, poziție în clasament, podiumuri, victorii și număr de curse acumulate în sezonul curent până înaintea cursei prezise, reflectând competitivitatea de ansamblu din sezon.
- **Forța echipei** — forma și clasamentul echipei surprind faptul că performanța în Formula 1 depinde decisiv de monopost, nu doar de pilot.

- **Istoricul pe circuit** — media de sosire și rata de podium ale pilotului și echipei pe circuitul respectiv, din toți anii anteriori, modelând afinitatea cu un anumit traseu.
- **Context** — experiența (numărul de curse din carieră), runda din sezon, prezența unei curse de sprint și condițiile meteo, factori care nuanțează predicția.

3.6.1 Principiul cauzal și starea istorică

Principiul cauzalității, esențial pentru evitarea scurgerii de informație (Secțiunea 2.8), este implementat printr-o procedură în două faze, repetată pentru fiecare cursă parcursă în ordine cronologică. În prima fază se calculează toate caracteristicile cursei curente folosind *exclusiv* starea istorică acumulată din cursele anterioare; abia în a doua fază, după ce caracteristicile au fost înregistrate, starea este actualizată cu rezultatul cursei curente. Astfel, este imposibil ca rezultatul unei curse să influențeze propriile sale caracteristici. Starea istorică este menținută în structuri de date eficiente: cozi cu lungime fixă (eng. *deque*) pentru mediile rulante pe ultimele $N = 5$ curse, dicționare de sume pentru statisticile sezoniere (resetate la schimbarea sezonului) și liste pe perechea (pilot, circuit) pentru istoricul pe traseu. Listarea 3.1 ilustrează această ordine pentru caracteristica “puncte acumulate în sezon”: valoarea atribuită cursei curente reflectă doar cursele anterioare.

```

1 # Pas 1: caracteristicile cursei R folosesc DOAR starea de pana acum.
2 row["season_points"] = state_points[driver]      # puncte înainte de cursa R
3
4 # Pas 2: abia DUPA ce s-au calculat caracteristicile,
5 #         starea este actualizata cu rezultatul cursei R.
6 state_points[driver] += points_in_race_R

```

Listarea 3.1: Actualizarea stării istorice după calcularea caracteristicilor (principiul anti-leakage).

Tabelul 3.5 ilustrează numeric acest mecanism pentru un pilot care obține constant 25 de puncte pe cursă (câștigă fiecare cursă). Valorile caracteristicilor atribuite fiecărei runde reflectă *exclusiv* cursele anterioare: înaintea primei runde, pilotul nu are istoric, deci primește valori implicite (poziție medie de sosire 15, zero puncte de sezon); înaintea celei de-a doua runde, caracteristicile încorporează rezultatul primei curse (medie de sosire 1, 25 de puncte), și așa mai departe. Crucial, punctele de sezon înaintea rundei a doua sunt 25, nu 50 — dovada că rezultatul cursei curente nu se scurge în propriile sale caracteristici.

Tabelul 3.5 Trasarea caracteristicilor cauzale pentru un pilot care câștigă fiecare cursă.

Rundă	form_avg_finish	season_points	career_races
1	15 (implicit)	0	0
2	1	25	1
3	1	50	2

3.6.2 Valori implicite pentru piloți și echipe fără istoric

O dificultate practică apare la *debutanți* și la prima apariție a unui pilot pe un circuit: în aceste cazuri starea istorică este goală, iar mediile rulante nu pot fi calculate. Soluția adoptată este atribuirea unor valori implicite rezonabile, care plasează un competitor necunoscut spre coada plutonului (de exemplu, o poziție medie de sosire de 15 și o rată de podium nulă). Această imputare deterministă este preferabilă eliminării exemplurilor incomplete (care ar reduce setul de date și ar introduce un dezechilibru artificial) și menține caracterul cauzal, întrucât valorile implicite nu folosesc nicio informație din viitor. Tot în această fază se face și o curățare defensivă a valorilor anormale — de pildă, o poziție pe grilă egală cu zero, semnalând o plecare din pit lane, este înlocuită cu valoarea implicită pentru grilă.

3.6.3 Inventarul complet al caracteristicilor

Pentru transparență și reproductibilitate, Tabelul 3.6 enumeră toate cele 29 de caracteristici, împreună cu o descriere succintă și cu valoarea implicită folosită în absența istoricului. Caracteristicile marcate cu “—” în coloana de valori implicite sunt derivate direct din datele cursei curente (de exemplu, indicatorii binari calculați din poziția pe grilă) și nu necesită imputare.

3.6.4 Eticheta și matricea de caracteristici

În încheierea acestei etape se adaugă *eticheta* de antrenare, *is_podium*, definită ca 1 dacă poziția finală este ≤ 3 și 0 în caz contrar (Secțiunea 2.2). Pe ansamblul celor 3458 de exemple, eticheta are media 0,150, confirmând cantitativ dezechilibrul de clase discutat teoretic în Secțiunea 2.4: doar circa 15% dintre perechile cursă–pilot corespund unui podium. Setul de caracteristici, împreună cu eticheta, este materializat într-o matrice numerică $\mathbf{X} \in \mathbb{R}^{n \times 29}$ și un vector $\mathbf{y}$, formatul standard consumat de toate cele trei modele.

3.7 Implementarea modelelor

Cele trei modele sunt implementate de la zero, peste NumPy, respectând o interfață comună care le face interschimbabile în restul sistemului. Această uniformitate este esențială pentru studiul comparativ: procedura de validare, predicția și evaluarea funcționează identic, indiferent de model, ceea ce garantează că diferențele de performanță observate provin din algoritmi, nu din condiții experimentale diferite.

Tabelul 3.6 Inventarul complet al celor 29 de caracteristici cauzale.

Caracteristică	Descriere	Implicit
grid	poziția pe grila de start	20
quali_pos	poziția în calificări	20
is_pole	1 dacă pleacă din pole position	—
start_top3	1 dacă pleacă din primele 3 poziții	—
start_top10	1 dacă pleacă din primele 10 poziții	—
form_avg_finish	media pozițiilor de sosire (ultimele 5 curse)	15
form_avg_points	media punctelor (ultimele 5 curse)	0
form_podium_rate	rata de podium (ultimele 5 curse)	0
form_dnf_rate	rata de abandon (ultimele 5 curse)	0,1
form_avg_grid	media pozițiilor pe grilă (ultimele 5 curse)	15
last_finish	poziția de sosire în ultima cursă	15
season_points	punctele acumulate în sezon (până la cursă)	0
season_position	poziția în clasamentul sezonului	20
season_podiums	numărul de podiumuri în sezon	0
season_wins	numărul de victorii în sezon	0
season_races	numărul de curse disputate în sezon	0
team_form_avg_finish	forma echipei: media de sosire	12
team_form_avg_points	forma echipei: media punctelor	0
team_season_points	punctele echipei în sezon	0
team_season_position	poziția echipei în clasament	10
circuit_driver_avg_finish	media de sosire a pilotului pe circuit	15
circuit_driver_podium_rate	rata de podium a pilotului pe circuit	0
circuit_team_avg_finish	media de sosire a echipei pe circuit	12
career_races	numărul total de curse din carieră	0
round	runda din sezon	—
is_sprint_weekend	1 dacă weekendul include o cursă de sprint	—
rain	1 dacă a plouat în cursă	0
track_temp	temperatura medie a pistei (°C)	30
air_temp	temperatura medie a aerului (°C)	22

3.7.1 Contractul comun și standardizarea

Toate modelele moștenesc dintr-o clasă de bază care fixează un contract minimal: metoda `fit(X, y)` antrenează modelul, `predict_proba(X)` întoarce probabilitatea de podium pentru fiecare exemplu, iar `predict(X)` aplică pragul de 0,5 pentru a obține eticheta binară. Cheia întregii soluții este `predict_proba`: probabilitatea de podium servește drept scor de ordonare a piloților (Secțiunea 3.8), nu doar drept etichetă. Pe lângă contract, clasa de bază expune și importanța caracteristicilor, atunci când modelul o oferă.

Tot aici este implementată *standardizarea* (z-score) folosită de regresia logistică: media și abaterea standard se învață *doar* pe datele de antrenament (metoda `fit`) și se reaplică identic la validare și test (metoda `transform`), ca în Listarea 3.2. Această disciplină — a învăța parametrii de preprocesare exclusiv pe antrenament — previne scurgerea prin preprocesare (Secțiunea 2.8) și este respectată riguros în implementare. Coloanele constante (abatere standard nulă) sunt tratate special, pentru a evita împărțirea la zero.

```
1 def fit(self, X):
2     self.mean_ = X.mean(axis=0)
3     std = X.std(axis=0)
4     std[std == 0] = 1.0          # evita impartirea la zero pe coloane
    constante
5     self.std_ = std
6     return self
7
8 def transform(self, X):
9     return (X - self.mean_) / self.std_    # reaplica parametrii invatati pe
    train
```

Listarea 3.2: Standardizarea z-score, cu parametri învățați doar pe antrenament.

3.7.2 Regresia logistică

Regresia logistică (Secțiunea 2.5) a fost implementată prin coborâre pe gradient în varianta *batch*, cu regularizare L_2 și ponderare pe clase. Caracteristicile sunt standardizate intern, parametrii standardizării fiind învățați exclusiv pe datele de antrenament. Un detaliu de implementare important este folosirea unei variante *numeric stabile* a funcției sigmoid, care tratează separat argumentele pozitive și negative pentru a evita depășirea de domeniu (eng. *overflow*) la exponențiere. Listarea 3.3 prezintă bucla de antrenare, care implementează ecuațiile (2.5) și (2.6) și înregistrează istoricul funcției de cost pentru analiza convergenței.

```
1 for _ in range(self.n_iters):
2     z = X @ self.w + self.b
3     p = _sigmoid(z)
4     # Pierdere (cross-entropy ponderata + L2) la pasul curent.
5     ce = -(sample_w * (y*np.log(p+eps) + (1-y)*np.log(1-p+eps))).sum() /
    sw_sum
```

```

6     self.loss_history_.append(float(ce + 0.5*self.l2 * float(self.w @
self.w)))
7
8     err = (p - y) * sample_w                # gradientul cross-entropy,
ponderat
9     grad_w = X.T @ err / sw_sum + self.l2 * self.w
10    grad_b = err.sum() / sw_sum
11    self.w -= self.lr * grad_w
12    self.b -= self.lr * grad_b

```

Listarea 3.3: Bucla de coborare pe gradient a regresiei logistice.

În cod, vectorul `sample_w` conține ponderile pe exemple care contracarează dezechilibrul de clase: fiecare exemplu primește o pondere invers proporțională cu frecvența clasei sale, astfel încât cele circa 15% de exemple de podium să cântărească la fel de mult, în total, ca restul de 85%. Întreaga buclă este *vectorizată*: produsele matrice–vector $\mathbf{X}\mathbf{w}$ și $\mathbf{X}^T \mathbf{e}$ procesează toate exemplele simultan, fără cicluri Python explicite, ceea ce face antrenarea rapidă chiar și pentru mii de iterații. Memorarea istoricului funcției de cost permite verificarea ulterioară a convergenței — curba de cost trebuie să scadă monoton și să se aplatizeze, semn că pasul de învățare a fost ales adecvat.

3.7.3 Arborele de decizie

Arborele de decizie (Secțiunea 2.6) folosește criteriul entropie plus câștig de informație. Structura arborelui este reprezentată prin noduri ușoare, fiecare reținând fie testul (caracteristica și pragul) și cei doi copii, fie, în cazul frunzelor, probabilitatea de podium — fracția de exemple pozitive ajunse în acea frunză. Construirea este recursivă, iar oprirea unui nod se produce la atingerea adâncimii maxime, a unui număr insuficient de exemple sau a unui nod deja pur.

Provocarea de eficiență a arborelui este căutarea celei mai bune divizări, care, naiv implementată, ar testa fiecare prag posibil pentru fiecare caracteristică — un cost prohibitiv. Soluția adoptată *vectorizează* această căutare: pentru o caracteristică, valorile sunt sortate o singură dată, iar prin sume cumulative ale etichetelor se obține instantaneu numărul de exemple pozitive din partea stângă pentru *fiecare* prag posibil, deci entropiile ambilor copii pentru toate pragurile candidate simultan. Se evaluează doar pragurile valide — acolo unde valoarea caracteristicii se schimbă efectiv și unde ambii copii respectă numărul minim de exemple pe frunză. Listarea 3.4 prezintă nucleul acestui calcul, corespunzând ecuațiilor (2.7) și (2.8).

```

1 cum_pos      = np.cumsum(y_sorted)                # pozitive cumulate in stanga
2 left_total   = np.arange(1, n)                    # numar exemple in stanga
3 left_pos     = cum_pos[:-1]
4 right_total  = n - left_total
5 right_pos    = total_pos - left_pos
6
7 h_left       = _binary_entropy(left_pos / left_total)
8 h_right      = _binary_entropy(right_pos / right_total)
9 child_entropy = (left_total/n)*h_left + (right_total/n)*h_right
10 gain        = parent_entropy - child_entropy      # castigul de informatie

```

Listarea 3.4: Calculul vectorizat al castigului de informatie pentru toate pragurile unei caracteristici.

Un exemplu numeric clarifică acest calcul. Fie un nod cu șase exemple, dintre care trei pozitive și trei negative, perfect separabile printr-o caracteristică (de pildă, valorile 0; 0,1; 0,2 sunt negative, iar 1,0; 1,1; 1,2 sunt pozitive). Entropia părintelui este $H(3/6) = 1$ bit (impuritate maximă). Un prag plasat între 0,2 și 1,0 produce un copil stâng cu trei exemple negative ($H = 0$) și un copil drept cu trei exemple pozitive ($H = 0$), deci o entropie a copiilor nulă. Câștigul de informație este, conform ecuației (2.8), $IG = 1 - 0 = 1$ bit — valoarea maximă posibilă, semnalând o divizare perfectă. Acest scenariu este folosit și ca test unitar al criteriului (Secțiunea 3.13).

Construirea propriu-zisă a arborelui este recursivă (Listarea 3.5): un nod devine frunză dacă s-a atins adâncimea maximă, dacă are prea puține exemple sau dacă este deja pur (toate exemplele din aceeași clasă); altfel, se caută cea mai bună divizare, iar exemplele sunt împărțite în doi copii pe baza pragului ales. La predicție, fiecare exemplu este rutat recursiv prin arbore, de la rădăcină spre frunza corespunzătoare, partiționarea fiind realizată tot vectorizat, pe măști booleene, pentru eficiență.

```
1 node.value = pos / n # P(podium) in nod = fractia de pozitive
2 if (depth >= self.max_depth or n < self.min_samples_split
3     or pos == 0 or pos == n): # nod pur sau prea mic -> frunza
4     node.is_leaf = True
5     return node
6
7 feat, thr, gain = self._best_split(X, y) # cea mai buna divizare
8 (vectorizat)
9 if feat is None or gain <= 0:
10     node.is_leaf = True
11     return node
12
13 mask = X[:, feat] <= thr
14 node.left = self._build(X[mask], y[mask], depth + 1) # recursiv stanga
15 node.right = self._build(X[~mask], y[~mask], depth + 1) # recursiv dreapta
```

Listarea 3.5: Condițiile de oprire și recursivitatea construirii arborelui.

Arborele suportă, în plus, un parametru `max_features` care limitează numărul de caracteristici candidate la fiecare divizare la un subset aleator — mecanism nefolosit de arborele individual, dar esențial pentru pădurea aleatoare. Pe parcursul construirii, fiecare divizare acumulează o contribuție la *importanța caracteristicilor*, egală cu câștigul de informație ponderat cu numărul de exemple din nod; la final, aceste contribuții sunt normalizate, oferind o măsură de tip reducere medie de impuritate (Secțiunea 2.7).

3.7.4 Pădurea aleatoare

Pădurea aleatoare (Secțiunea 2.7) este construită peste arborele de decizie propriu, ceea ce ilustrează un principiu de proiectare prin compoziție: ansamblul reutilizează integral implementarea arbore-

lui, adăugând doar logica de eşantionare şi agregare. Fiecare arbore primeşte un eşantion bootstrap al rândurilor (extragere cu revenire) şi un generator de numere aleatoare propriu, derivat dintr-un generator-părinte. Această schemă de generatoare imbricate asigură o proprietate aparent contradic-torie, dar necesară: arborii sunt *diferiți* între ei (fiecare cu propria sursă de aleator, deci decorelați), dar întregul proces este *reproductibil* (pornind de la aceeaşi sămânţă, se obține aceeaşi pădure). Pre-dicția ansamblului este media probabilităților arborilor, iar opțional se poate calcula scorul out-of-bag (Secțiunea 2.7). Listarea 3.6 prezintă bucla de antrenare a ansamblului.

```

1 for _ in range(self.n_estimators):
2     idx = rng.integers(0, n, size=n)          # esantion bootstrap (cu revenire)
3     tree = DecisionTree(
4         max_depth=self.max_depth,
5         min_samples_leaf=self.min_samples_leaf,
6         max_features=self.max_features,      # subset aleator de caracteristici
7         rng=np.random.default_rng(rng.integers(0, 2**32 - 1)),
8     )
9     tree.fit(X[idx], y[idx])
10    self.trees.append(tree)

```

Listarea 3.6: Antrenarea pe baza de bagging a padurii aleatoare.

Opțional, în timpul antrenării se poate calcula scorul out-of-bag (Secțiunea 2.7): pentru fiecare arbore se identifică exemplele neselectate de eşantionul bootstrap, iar predicțiile acestora se acumulează doar din arborii care nu le-au “văzut” (Listarea 3.7). Media acestor predicții oferă o estimare a erorii de generalizare fără un set de validare separat, ceea ce constituie un avantaj practic notabil al pădurii aleatoare. Importanța caracteristicilor pentru pădure se obține mediind importanțele arborilor compo-nenți.

```

1 oob_mask = np.ones(n, dtype=bool)
2 oob_mask[idx] = False                      # exemplele NEselectate de acest
3                                           # arbore
4 if oob_mask.any():
5     oob_sum[oob_mask] += tree.predict_proba(X[oob_mask])
6     oob_count[oob_mask] += 1                # de cati arbori a fost "vazut"
7                                           # fiecare
8 # La final: predictia OOB = oob_sum / oob_count, comparata cu y pentru scorul
9 # OOB.

```

Listarea 3.7: Acumularea predictiilor out-of-bag (OOB) pe mostrele neselectate.

3.7.5 Complexitatea computațională

Cele trei modele diferă semnificativ ca cost de antrenare. Regresia logistică are o complexitate de $O(T \cdot n \cdot d)$, unde T este numărul de iterații, n numărul de exemple și d numărul de caracteristici, fiecare iterație fiind o trecere vectorizată peste toate datele. Arborele de decizie costă aproximativ $O(d \cdot n \log n)$ per nivel, dominat de sortarea valorilor la căutarea divizărilor; vectorizarea prin sume cumulative reduce constanta acestui cost. Pădurea aleatoare multiplică costul arborelui cu numărul de

arbori, dar fiecare arbore lucrează pe un subset de $\sqrt{d}$ caracteristici per divizare, ceea ce atenuează creșterea. În practică, pe setul de date folosit, toate cele trei modele se antrenează în ordinul secundelor, iar timpul mediu de antrenare per fold este înregistrat de procedura de validare pentru comparație (Secțiunea 3.9).

3.7.6 Fabrica de modele și hiperparametrii

Pentru a uniformiza construirea modelelor, o funcție-fabrică creează modelul potrivit pornind de la o cheie scurtă (`logreg`, `tree` sau `forest`) și de la hiperparametrii din fișierul de configurare. Acest mecanism centralizează valorile hiperparametrilor și permite, în același timp, suprascrierea lor punctuală în experimente. Hiperparametrii sunt ficși pe tot parcursul studiului comparativ (Tabelul 3.7), pentru ca diferențele de performanță să reflecte algoritmii, nu o reglare fină inegală. Valorile au fost alese pe baza recomandărilor uzuale din literatură (Capitolul 2): un pas de învățare moderat și un număr mare de iterații pentru convergența regresiei logistice; o adâncime limitată pentru arbore, care îi controlează varianța; și o pădure cu suficient de mulți arbori și $\sqrt{d}$ caracteristici per divizare, conform practicii consacrate pentru clasificare.

Tabelul 3.7 Hiperparametrii ficși ai celor trei modele.

Model	Hiperparametru	Valoare
Regresie logistică	pas de învățare η	0,1
	număr de iterații	3000
	regularizare L_2 (λ)	0,01
	ponderare pe clase	activată
Arbore de decizie	adâncime maximă	8
	minim exemple pentru divizare	10
	minim exemple pe frunză	5
Random Forest	număr de arbori	60
	adâncime maximă	12
	caracteristici per divizare	$\sqrt{d}$
	minim exemple pe frunză	4

3.8 Predicția podiumului și agregarea în campionat

3.8.1 Ordonarea piloților per cursă

Un clasificator binar aplicat independent fiecărui pilot nu garantează exact trei predicții pozitive pe cursă: în funcție de prag, pot rezulta doi sau cinci piloți “de podium”, ceea ce nu corespunde realității în care podiumul are exact trei locuri. De aceea, podiumul este determinat prin *ordonare* (Secțiunea 2.2): pentru fiecare cursă se calculează probabilitatea de podium a tuturor piloților, aceștia sunt

sortați descrescător, iar primii trei formează podiumul prezis (probabilitatea cea mai mare corespunde locului P1). Egalitățile de probabilitate se departajează după poziția pe grilă — un criteriu obiectiv, cunoscut înainte de cursă, care favorizează pilotul plecat mai în față. Listarea 3.8 prezintă această ordonare.

```
1 # Sortare descrescătoare după probabilitate; egalitatile -> după grid (asc).
2 out = race_df[cols].copy()
3 out["proba"] = np.asarray(proba, dtype=float)
4 out = out.sort_values(["proba", "grid"], ascending=[False,
5     True]).reset_index(drop=True)
6 out["pred_position"] = np.arange(1, len(out) + 1) # 1 = P1, 2 = P2, ...
```

Listarea 3.8: Ordonarea pilotilor unei curse după probabilitatea de podium.

Pentru a ilustra concret mecanismul, Tabelul 3.8 prezintă primii șase piloți ordonați de pădurea aleatoare pentru o cursă reală (Marele Premiu al Australiei 2025, prima rundă a sezonului), alături de probabilitatea de podium și de poziția finală reală. Podiumul prezis (primii trei după probabilitate) este NOR, PIA și VER; podiumul real a fost NOR, VER și RUS. Exemplul evidențiază atât puterea, cât și limitele metodei: două dintre cele trei poziții sunt nimerite (NOR și VER), însă PIA, deși clasat al doilea ca probabilitate, a terminat abia pe locul nouă, în timp ce RUS, cu o probabilitate mai mică, a urcat pe podium — o ilustrare a incertitudinii inerente predicției sportive.

Tabelul 3.8 Exemplu de ordonare a piloților după probabilitatea de podium (Australia 2025, runda 1).

Loc prezis	Pilot	Grilă	$\hat{p}$ (podium)	Poziție reală
1	NOR	1	0,652	1
2	PIA	2	0,565	9
3	VER	3	0,522	2
4	RUS	4	0,432	3
5	LEC	7	0,337	8
6	HAM	8	0,316	10

3.8.2 Agregarea în clasament de campionat

Aceeași ordonare per cursă stă la baza unui clasament de campionat estimat la finalul sezonului. Pentru fiecare cursă, clasamentul prezis complet (nu doar primii trei) primește punctele standard din Formula 1 (25 pentru locul întâi, 18 pentru al doilea, 15 pentru al treilea și așa mai departe, până la un punct pentru locul al zecelea), iar punctele se însumează pe tot sezonul. Listarea 3.9 schițează acest calcul. Clasamentul real se obține analog, însumând punctele efective, iar comparația dintre cele două clasamente oferă o evaluare la nivel de sezon, complementară celei per cursă.

```
1 # Pentru fiecare cursă: puncte F1 după clasamentul prezis, însumate pe sezon.
2 for _, race in df.groupby("round", sort=True):
3     ranked = rank_drivers(race, race["_proba"].to_numpy())
4     for _, row in ranked.iterrows():
5         pos = int(row["pred_position"])
```

```

6 points[driver] += F1_POINTS.get(pos, 0)      # doar top 10 puncteaza

```

Listarea 3.9: Agregarea probabilitatilor intr-un clasament de campionat prezis.

3.9 Validarea temporală pe ani

Procedura de validare implementează schema descrisă teoretic în Secțiunea 2.9 și reprezintă mecanismul prin care lucrarea garantează o estimare onestă a performanței. Ideea fundamentală este că modelul nu trebuie evaluat niciodată pe date pe care le-a “văzut” în vreo formă la antrenare. Listarea 3.10 prezintă bucla principală: pentru fiecare sezon de test S , modelul este reantrenat *de la zero* pe sezoanele până la $S - 2$, sezonul $S - 1$ este ținut deoparte ca validare, iar performanța finală se măsoară pe sezonul de test S , complet nevăzut. Reantrenarea de la zero la fiecare pas, în loc de o actualizare incrementală, este o alegere deliberată: ea garantează că modelul evaluat pe sezonul S nu păstrează nicio urmă din sezoanele ulterioare.

```

1 for i in range(2, len(seasons)):          # avem nevoie de S-1 si de un
    sezon inainte de el
2     test_season = seasons[i]              # S
3     val_season = seasons[i - 1]           # S-1 (validare)
4     train = feats[feats["season"].isin(seasons[:i - 1])] # <= S-2
5     val = feats[feats["season"] == val_season]
6     test = feats[feats["season"] == test_season]
7
8     model = build_model(model_key, **overrides)
9     model.fit(*feature_matrix(train)[:2]) # antrenare doar pe <= S-2
10    proba_val = model.predict_proba(feature_matrix(val)[0]) # validare pe
    S-1
11    proba_test = model.predict_proba(feature_matrix(test)[0]) # test pe S

```

Listarea 3.10: Bucla de validare temporală cu separare train / validare / test pe ani.

Bucla pornește de la al treilea sezon disponibil, deoarece schema necesită cel puțin un sezon de antrenament ($\leq S - 2$) și un sezon de validare ($S - 1$). Cu cele opt sezoane disponibile (2018–2025), rezultă șase sezoane de test (2020–2025), fiecare cu propriul model antrenat doar pe trecut. Pentru fiecare sezon de test se calculează și se rețin cele patru metrici, atât pe sezonul de test, cât și pe cel de validare; raportarea separată a validării, deși nu este folosită la reglarea hiperparametrilor (care sunt ficși), servește transparenței metodologice, arătând că performanța pe sezonul ținut deoparte este coerentă cu cea de test. Rezultatele individuale sunt agregate într-o structură care expune atât tabelul per sezon, cât și mediile globale și acuratețea ori scorul F_1 pe toate exemplele de test reunite, plus timpul mediu de antrenare per fold — util pentru compararea costului computațional al celor trei modele.

3.10 Antrenarea incrementală in-sezon

Pe lângă validarea pe ani, soluția include o analiză a efectului datelor *in-sezon*: în practică, atunci când prezicem o cursă din mijlocul unui sezon, dispunem deja de rezultatele curselor anterioare din același sezon. Pentru a cuantifica acest avantaj, pentru fiecare rundă R a unui sezon se compară două regimuri de antrenare, ambele prezicând aceeași cursă R :

- **fără in-sezon** — model antrenat doar pe sezoanele anterioare;
- **cu in-sezon** — model antrenat pe sezoanele anterioare plus cursele $1, \dots, R - 1$ din sezonul curent.

La prima rundă cele două regimuri coincid (nu există curse in-sezon anterioare), iar diferența crește pe măsură ce se acumulează date in-sezon. Justificarea acestei analize este practică: antrenarea “cu in-sezon” corespunde modului real de utilizare a unui predictor pe parcursul sezonului, în care fiecare cursă nou disputată devine date de antrenament pentru următoarea. Compararea cu regimul “fără” cuantifică, astfel, câștigul adus de actualizarea modelului cu informația cea mai recentă.

Din punct de vedere al implementării, ambele regimuri respectă strict cauzalitatea: modelul “cu in-sezon” pentru runda R este antrenat exclusiv pe sezoanele anterioare plus rundele $1, \dots, R - 1$, nicio dată pe runda R însăși. Modelul de bază (“fără”) se antrenează o singură dată pe sezoanele anterioare și se reutilizează la toate rundele, în timp ce modelul augmentat se reantrenează la fiecare rundă pentru a încorpora cursele in-sezon acumulate. Deoarece metricile calculate pe o singură cursă sunt grosiere (o cursă are un singur podium, deci valorile sunt fracții cu numitor mic), rezultatele se raportează ca medie cumulativă pe runde, care netezește fluctuațiile (Secțiunea 4.7).

3.11 Metrici și instrumentația experimentală

Toate metricile din Secțiunea 2.10 sunt implementate manual, peste NumPy, fără biblioteci de învățare automată, în acord cu cerința de implementare de la zero. Metricile de clasificare (acuratețe, precizie, recall, F_1) se derivă direct din matricea de confuzie, în timp ce metricile specifice problemei se calculează la nivel de cursă: pentru fiecare cursă se compară podiumul prezis (primii trei după probabilitate) cu cel real (primii trei după poziția finală), iar rezultatele se mediază pe sezon. Listarea 3.11 prezintă nucleul acestui calcul.

```
1 for _, race in df.groupby("round", sort=True):
2     actual = list(race.sort_values("position")["driver"].head(3))    # podium
   real
3     ranked = rank_drivers(race, race["_proba"].to_numpy())
4     pred    = list(ranked["driver"].head(3))                        # podium
   prezis
5
```



```

6     inter = len(set(pred) & set(actual))
7     hit.append(inter / 3)                                # rata de nimerire (0,
1/3, 2/3, 1)
8     exact_set.append(1.0 if set(pred) == set(actual) else 0.0) #
podium exact
9     winner.append(1.0 if pred[0] == actual[0] else 0.0)    #
castigator Top-1

```

Listarea 3.11: Calculul metricilor de podium per cursa, apoi mediate pe sezon.

Pentru fiecare sezon de test, sistemul salvează rezultatele atât tabelar (fișiere CSV cu metricile per sezon și un sumar comparativ), cât și sub formă de grafice generate cu matplotlib: câte un grafic pe metrică, defalcat pe sezoanele de test, un grafic-sumar comparativ între modele și graficele de importanță a caracteristicilor. Această dublă raportare — numerică și vizuală — facilitează atât verificarea exactă, cât și interpretarea de ansamblu din Capitolul 4.

3.12 Interfața în linie de comandă

Toate funcționalitățile sunt expuse printr-o interfață unică în linie de comandă, organizată pe subcomenzi (Tabelul 3.9). Această alegere face soluția ușor de rulat și de reprodus: un flux complet de lucru constă din construirea datasetului, urmată de evaluarea comparativă sau de predicția unei curse ori a unui sezon. Subcomenzile partajează aceeași logică de antrenare strict cauzală — de exemplu, predicția unei curse din mijlocul sezonului antrenează automat modelul doar pe trecutul acelei curse, ilustrând în practică principiul anti-leakage.

Tabelul 3.9 Subcomenzile interfeței în linie de comandă.

Subcomandă	Funcție
build-data	construiește tabelul de rezultate și caracteristicile din cache
train	antrenează și salvează un model până la un sezon-prag
evaluate	validare temporală pe ani + tabele și grafice comparative
predict-race	podiumul prezis al unei curse
predict-season	podium per cursă + clasament de campionat prezis
insezon	compară performanța cu și fără cursele in-sezon

3.12.1 Fluxul de lucru complet

Un experiment complet se desfășoară într-o succesiune naturală de comenzi. Mai întâi, datasetul este construit o singură dată din cache, materializând tabelul de rezultate și caracteristicile. Apoi, studiul comparativ al celor trei modele se obține printr-o singură comandă de evaluare, care produce tabelele și graficele din Capitolul 4. În fine, soluția poate fi folosită demonstrativ pentru a prezice o cursă anume sau un întreg sezon. Listarea 3.12 prezintă această secvență.

```

1 python main.py build-data # 1. construiește
   datasetul
2 python main.py evaluate --model all # 2. studiu comparativ
   + grafice
3 python main.py predict-race --season 2025 --round 1 --model forest # 3. o
   cursa
4 python main.py predict-season --season 2025 --model forest # 4. un
   sezon
5 python main.py insezon --season 2025 --model all # 5. analiza in-sezon

```

Listarea 3.12: Fluxul de lucru complet, exprimat prin comenzile interfetei.

Fiecare comandă de predicție reantrenează automat modelul doar pe trecutul corespunzător, astfel încât principiul anti-leakage este respectat în mod transparent pentru utilizator, fără pași manuali suplimentari.

3.13 Testarea automată și verificarea corectitudinii

Deoarece algoritmi sunt implementați de la zero, corectitudinea lor nu poate fi presupusă, ci trebuie verificată explicit. De aceea, soluția include o suită de teste automate, organizată pe patru module, care acoperă punctele critice ale implementării. Rolul lor este dublu: validează corectitudinea funcțională și, mai ales pentru testele anti-leakage, protejează cea mai importantă proprietate metodologică a lucrării.

Primul modul testează *absența scurgerii de informație*. Pe un mini-sezon sintetic, cu rezultate cunoscute, se verifică faptul că, la prima cursă, toate caracteristicile au valori implicite (nu există istoric), iar punctele de sezon înaintea unei curse reflectă exact suma curselor *anterioare*, niciodată cursa curentă. Listarea 3.13 prezintă acest test: un pilot care obține 25 de puncte pe cursă trebuie să aibă, înaintea celei de-a doua curse, 25 de puncte (nu 50) și, înaintea celei de-a treia, 50. Acest test transformă principiul causal dintr-o intenție într-o garanție verificabilă mecanic.

```

1 def test_season_points_exclude_current_race():
2     feats = add_features(_mini_long()) # AAA ia 25p/cursa
3     aaa = feats[feats["driver"] == "AAA"].sort_values("round")
4     # Inainte de R2 are 25 (din R1), nu 50; inainte de R3 are 50.
5     assert list(aaa["season_points"]) == [0.0, 25.0, 50.0]

```

Listarea 3.13: Test anti-leakage: punctele de sezon exclud cursa curentă.

Al doilea modul testează cei *trei algoritmi*, pe seturi-jucărie cu răspuns cunoscut: regresia logistică trebuie să separe aproape perfect date liniar separabile și să atribuie cel mai mare coeficient caracteristicii celei mai informative; arborele trebuie să învețe o regulă conjunctivă (de forma $x_0 > 0$ și $x_1 > 0$) și să acorde importanță aproape nulă unei caracteristici de zgomot; pădurea trebuie să generalizeze cel puțin la fel de bine ca un arbore pe date zgomotoase și să producă un scor out-of-bag rezonabil.

Al treilea modul verifică *metricile*, comparând valorile calculate cu rezultate derivate manual dintr-o matrice de confuzie cunoscută. Al patrulea modul testează *blocurile reutilizate* de analiza în-sezon, confirmând că regimul “cu în-sezon” folosește efectiv mai multe date decât cel de bază și că, la prima rundă, cele două regimuri coincid.

Această suită de teste oferă încredere că diferențele de performanță raportate în Capitolul 4 reflectă proprietățile reale ale algoritmilor, nu erori de implementare, și că rezultatele nu sunt afectate de scurgerea de informație.

3.14 Reproducibilitate

Reproducibilitatea a fost o cerință de proiectare urmărită pe tot parcursul soluției și se sprijină pe trei mecanisme. În primul rând, sursele de aleator sunt controlate printr-o sămânță fixă: pădurea aleatoare pornește de la un generator inițializat determinist, astfel încât două rulări produc exact aceeași pădure. În al doilea rând, accesul la date este strict offline, din cache (Secțiunea 3.3), ceea ce decuplează rezultatele de starea curentă a serviciilor externe. În al treilea rând, rezultatele intermediare costisitoare — tabelul de rezultate și matricea de caracteristici — sunt memorate pe disc în format Parquet și reutilizate, garantând că toate experimentele pornesc de la exact același set de date. Împreună, aceste mecanisme asigură că rezultatele raportate în Capitolul 4 pot fi regenerate identic.

3.15 Decizii de proiectare și compromisuri

Pe parcursul dezvoltării au fost luate mai multe decizii de proiectare, fiecare implicând un compromis pe care îl rezumăm aici, pentru a justifica forma finală a soluției.

Clasificare și ordonare, în loc de regresie pe poziție. Problema ar fi putut fi formulată ca o regresie directă a poziției finale. S-a preferat clasificarea binară a podiumului, urmată de ordonare (Secțiunea 3.8), deoarece produce o probabilitate ușor de interpretat și de agregat, iar obiectivul real — identificarea celor trei piloți de pe podium — este surprins mai natural de o problemă de ordonare decât de predicția exactă a fiecărei poziții.

Validare la granularitate de sezon, nu de cursă. O separare la nivel de cursă ar fi oferit mai multe puncte de evaluare, dar ar fi amestecat curse din același sezon între antrenament și test, slăbind separarea temporală. Granularitatea pe sezon (Secțiunea 3.9) este mai conservatoare și se aliniază cu ciclul real al competiției.

Coborâre pe gradient în varianta batch. Pentru regresia logistică s-a ales varianta batch, nu cea stocastică, întrucât volumul moderat al datelor permite treceri complete rapide, iar actualizările sunt mai stabile și mai ușor de analizat prin curba de cost.

Hiperparametri ficși, fără reglare automată. Reglarea hiperparametrilor pe sezonul de test ar fi introdus o formă subtilă de scurgere de informație, iar reglarea pe sezonul de validare ar fi complicat comparația. S-a optat pentru valori ficși, rezonabile (Tabelul 3.7), astfel încât diferențele de performanță să reflecte algoritmi, lăsând optimizarea automată ca direcție de dezvoltare ulterioară (Capitolul 5).

Imputarea cu valori implicite, în loc de eliminarea exemplilor. Exemplele fără istoric (debutanți) sunt completate cu valori implicite, nu eliminate, pentru a păstra toate cursele și a nu introduce un dezechilibru artificial; valorile implicite sunt deterministe și strict cauzale.

Aceste compromisuri reflectă prioritatea acordată rigorii metodologice și echității comparației, în detrimentul unei optimizări agresive a performanței absolute — în acord cu scopul declarat al lucrării, acela de studiu comparativ.

Capitolul 4

Rezultate

4.1 Configurația experimentală

Experimentele folosesc setul de date descris în Capitolul 3, cu validare temporală pe sezoanele de test 2020–2025. Conform schemei din Secțiunea 3.9, fiecare sezon S este prezis cu un model antrenat pe sezoanele până la $S - 2$, sezonul $S - 1$ fiind ținut deoparte ca validare. Rezultă șase sezoane de test, însumând 131 de curse, fiecare cu un model reantrenat de la zero exclusiv pe trecut. Numărul de exemple de antrenament crește de la 420 (pentru testul pe 2020) la 2500 (pentru testul pe 2025), pe măsură ce se acumulează sezoane.

Cele patru metrici raportate sunt acuratețea clasificării binare, rata de nimerire a podiumului, podiumul exact și acuratețea câștigătorului (Top-1), definite în Secțiunea 2.10. Condițiile de antrenare și evaluare sunt identice pentru toate cele trei modele, astfel încât diferențele observate să reflecte exclusiv algoritmi. Hiperparametrii sunt ficși pe tot parcursul studiului (Tabelul 3.7): pentru regresia logistică, pas de învățare 0,1, 3000 de iterații și regularizare L_2 cu $\lambda = 0,01$; pentru arbore, adâncime maximă 8; pentru pădure, 60 de arbori și adâncime maximă 12, cu $\sqrt{d}$ caracteristici per divizare. Pentru fiecare model se raportează atât mediile pe sezoane, cât și defalcarea pe fiecare sezon de test, alături de timpul de antrenare, măsurat pe parcursul validării.

4.2 Performanța comparativă

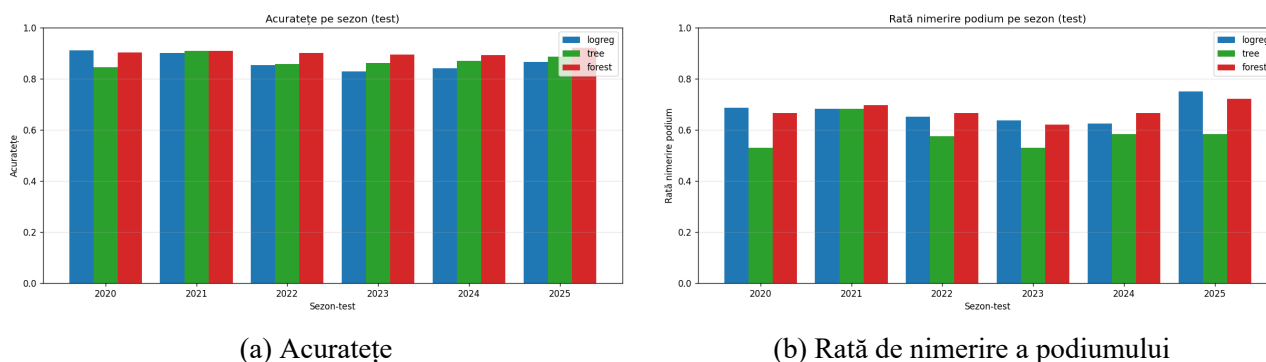
Tabelul 4.1 prezintă performanța medie a celor trei modele pe sezoanele de test, pe cele patru metrici. Pădurea aleatoare obține cea mai bună acuratețe (0,903), cea mai bună rată de nimerire a podiumului (0,673) și cel mai bun scor pentru podiumul exact (0,253), în timp ce regresia logistică conduce la cea mai bună acuratețe a câștigătorului (Top-1, 0,536). Arborele de decizie simplu este, în mod constant, modelul cel mai slab, ceea ce confirmă avantajul metodelor de ansamblu și al regularizării.

Tabelul 4.1 Performanța medie a modelelor pe sezoanele de test (validare temporală).

Model	Acuratețe	Rată podium	Podium exact	Câștigător (Top-1)
Regresie logistică	0,867	0,672	0,240	0,536
Arbore de decizie	0,872	0,581	0,167	0,388
Random Forest	0,903	0,673	0,253	0,471

Analiza tabelului relevă mai multe observații. În primul rând, pădurea aleatoare domină trei din cele patru metrici, dar cu marje modeste față de regresia logistică: diferența la rata de nimerire a podiumului este de doar 0,001, practic neglijabilă, în timp ce la acuratețe avantajul este de 0,036. Aceasta confirmă că, pe date tabelare, un model liniar simplu poate fi surprinzător de competitiv cu un ansamblu mult mai complex. În al doilea rând, regresia logistică conduce clar la acuratețea câștigătorului (Top-1, 0,536 față de 0,471 pentru pădure), sugerând că modelul liniar identifică mai bine favoritul fiecărei curse. În al treilea rând, arborele de decizie individual este în mod constant cel mai slab pe metricile de podium (rată 0,581, podium exact 0,167, Top-1 0,388), deși are o acuratețe comparabilă; acest contrast ilustrează limpede diferența dintre o acuratețe ridicată, ușor de obținut pe o clasă dezechilibrată, și capacitatea reală de a prezice podiumul. Rezultatul susține teoretic avantajul medierii din ansambluri, care reduce varianța mare a unui arbore unic (Secțiunea 2.7).

Figura 4.1 prezintă acuratețea și rata de nimerire a podiumului defalcate pe sezoanele de test, iar Figura 4.2 rezumă comparativ cele patru metrici (medii pe sezoane). Se observă că rata de nimerire a podiumului (Figura 4.1b) este în mod sistematic mai mică decât acuratețea (Figura 4.1a), aspect discutat în Secțiunea 4.5.

**Figura 4.1** Performanța pe sezoanele de test: acuratețe (a) și rată de nimerire a podiumului (b).

4.3 Variația performanței pe sezoane

Mediile din Tabelul 4.1 ascund o variabilitate notabilă de la un sezon la altul. Tabelul 4.2 prezintă defalcarea pe sezoane a celor patru metrici pentru pădurea aleatoare, modelul cu cea mai bună performanță medie. Acuratețea este remarcabil de stabilă (între 0,891 și 0,921), iar rata de nimerire a podiumului se menține în jurul valorii de 0,67. În schimb, acuratețea câștigătorului fluctuează puternic, de la 0,235 în 2020 la 0,773 în 2023.

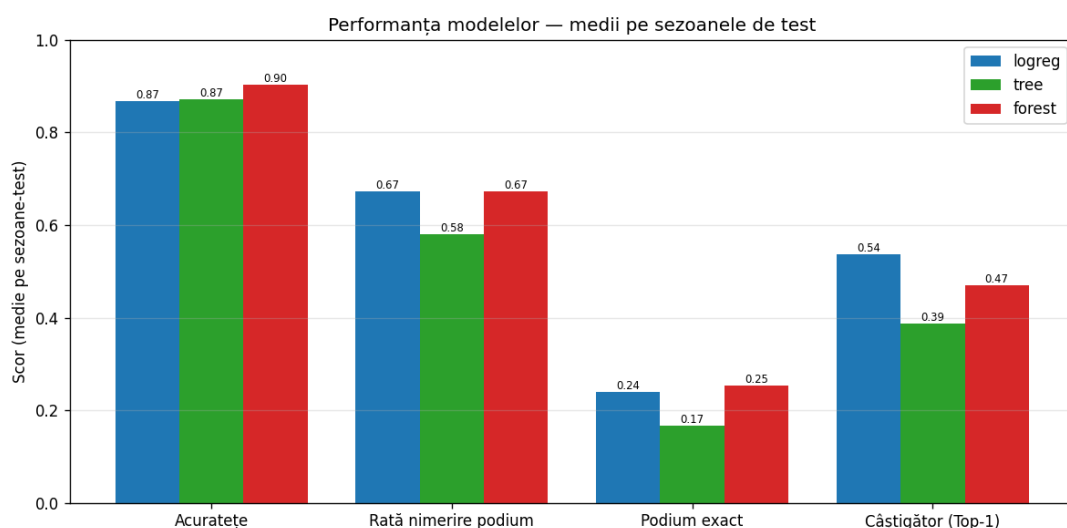


Figura 4.2 Performanța comparativă a celor trei modele pe cele patru metrici (medii pe sezoanele de test).

Tabelul 4.2 Performanța pădurii aleatoare pe fiecare sezon de test.

Sezon de test	2020	2021	2022	2023	2024	2025
Acuratețe	0,903	0,909	0,900	0,895	0,891	0,921
Rată podium	0,667	0,697	0,667	0,621	0,667	0,722
Podium exact	0,294	0,318	0,182	0,182	0,250	0,292
Câștigător (Top-1)	0,235	0,455	0,364	0,773	0,500	0,500

Această variație are o explicație legată de natura competiției. Acuratețea câștigătorului este ridicată în sezoanele dominate de un singur pilot — în 2023, un pilot a câștigat marea majoritate a curselor, ceea ce face favoritul ușor de prezis (toate modelele ating valori mari în acel sezon: 0,909 pentru regresia logistică și 0,773 pentru pădure). Dimpotrivă, în sezoanele echilibrate, identificarea câștigătorului exact devine mult mai grea. Faptul că metricile la nivel de cursă (podium exact, câștigător) fluctuează mai mult decât acuratețea de clasificare este de așteptat: ele se calculează pe un număr mic de curse (17–24 per sezon), deci sunt mai sensibile la evenimente individuale, precum abandonuri sau condiții meteo neobișnuite.

4.4 Costul computațional al antrenării

Pe lângă calitatea predicțiilor, studiul comparativ a măsurat și timpul de antrenare, înregistrat la fiecare reantrenare din procedura de validare. Tabelul 4.3 prezintă timpul mediu de antrenare per fold. Ordinea confirmă analiza de complexitate din Secțiunea 3.7.5: arborele de decizie este cel mai rapid, regresia logistică este de aproximativ trei ori mai lentă (din cauza celor 3000 de iterații de coborâre pe gradient), iar pădurea aleatoare este cea mai costisitoare, fiind în esență un multiplu al costului unui arbore. Chiar și așa, toate cele trei modele se antrenează sub o secundă în medie, ceea ce face întregul studiu rapid de reprodus.

Tabelul 4.3 Timpul mediu de antrenare per fold (validare temporală).

Model	Timp mediu de antrenare (s)
Arbore de decizie	0,05
Regresie logistică	0,17
Random Forest	0,82

Acest rezultat nuanțează compromisul performanță–cost: pădurea aleatoare oferă cea mai bună calitate medie, dar cu un cost de antrenare de circa șaisprezece ori mai mare decât al arborelui; regresia logistică se remarcă drept o alegere echilibrată, apropiată de pădure ca performanță, dar considerabil mai ieftină.

4.5 Discuție: acuratețea în prezența dezechilibrului

Un rezultat aparent paradoxal este faptul că acuratețea depășește 0,86 la toate modelele (Tabelul 4.1), în timp ce rata de nimerire a podiumului este de circa 0,67. Explicația rezidă în dezechilibrul de clase: deoarece doar aproximativ 15% dintre exemple sunt pozitive, un clasificator care prezice mereu “nu este podium” atinge deja o acuratețe de circa 85%. Prin urmare, acuratețea de 0,90 reprezintă o îmbunătățire modestă față de acest reper trivial, motiv pentru care lucrarea raportează ca metrici principale rata de nimerire a podiumului, podiumul exact și acuratețea câștigătorului, alături de acuratețea clasificării, în acord cu literatura privind clasele dezechilibrate [3], [4].

4.6 Importanța caracteristicilor

Dincolo de scorurile agregate, pădurea aleatoare oferă și un grad de interpretabilitate prin importanța caracteristicilor, calculată ca reducerea medie de impuritate (câștigul de informație) adusă de fiecare caracteristică în arborii ansamblului (Secțiunea 2.7). Tabelul 4.4 prezintă cele mai importante opt caracteristici pentru pădurea aleatoare, iar Figura 4.3 le ilustrează grafic.

Tabelul 4.4 Cele mai importante opt caracteristici pentru pădurea aleatoare.

Caracteristică	Categorie	Importanță
grid	grilă / calificări	0,143
quali_pos	grilă / calificări	0,110
team_form_avg_points	forța echipei	0,100
form_avg_points	formă pilot	0,074
team_form_avg_finish	forța echipei	0,072
start_top3	grilă / calificări	0,071
form_avg_grid	formă pilot	0,053
form_avg_finish	formă pilot	0,031

Rezultatul confirmă intuiția din domeniu și analiza exploratorie (Secțiunea 3.5): poziția de start — prin poziția pe grilă, poziția în calificări și indicatorul de plecare din primele trei — este cel mai puternic predictor, însumând circa o treime din importanța totală. Urmează *forța echipei* (forma de puncte și de sosire a echipei), reflectând faptul că în Formula 1 monopostul cântărește decisiv, apoi *forma recentă a pilotului*. Este de remarcat că modelul distribuie importanța pe mai multe caracteristici complementare, ceea ce explică robustețea sa.

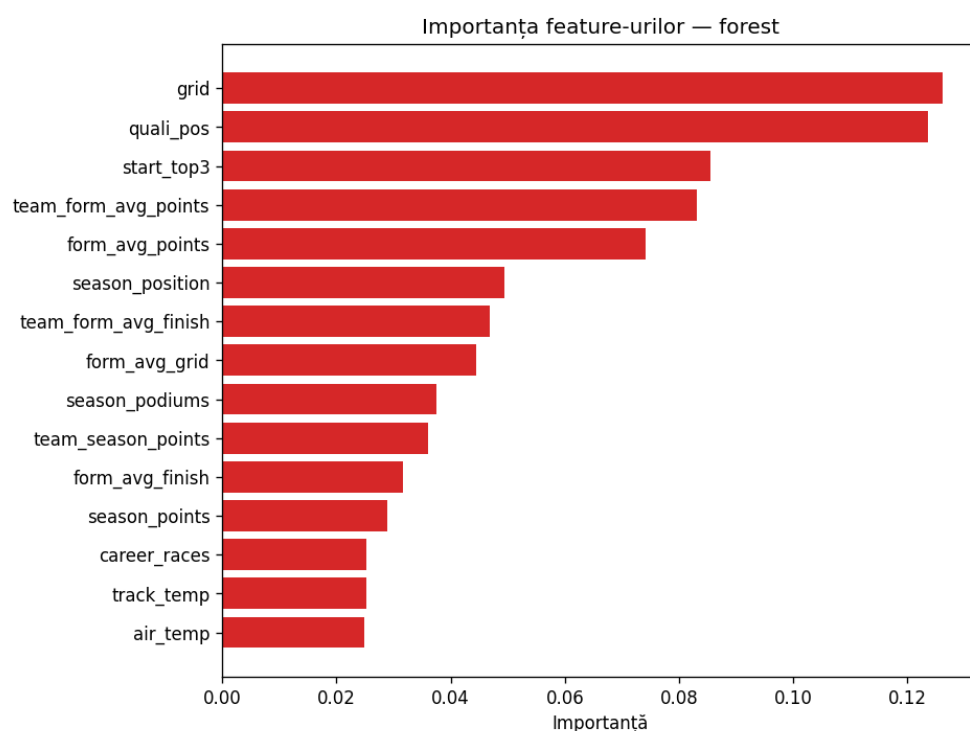


Figura 4.3 Importanța caracteristicilor pentru pădurea aleatoare (primele după importanță).

Modul în care fiecare model își distribuie importanța diferă semnificativ și explică, în parte, comportamentul lor. Arborele de decizie individual concentrează peste jumătate din importanță într-o singură caracteristică (poziția în calificări), ceea ce îl face fragil — o ilustrare directă a varianței mari a unui arbore unic. Regresia logistică, dimpotrivă, repartizează ponderi mari mai multor caracteristici legate de poziția de start și de clasament, comportament coerent cu natura sa liniară. Pădurea aleatoare se află la mijloc, combinând semnale multiple, ceea ce reflectă efectul de decorare al subspațiilor aleatoare de caracteristici (Secțiunea 2.7).

4.7 Efectul datelor in-sezon

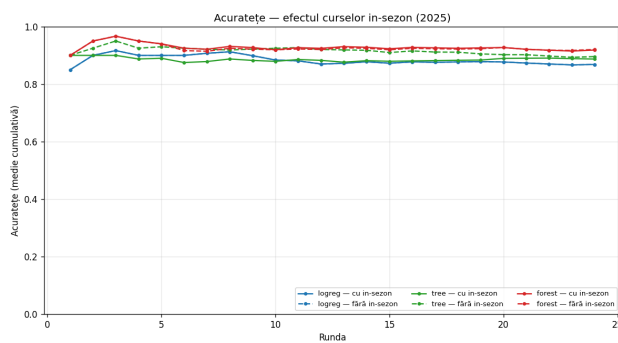
Conform metodologiei din Secțiunea 3.10, Tabelul 4.5 compară, pe sezonul 2025, performanța medie cu și fără cursele in-sezon în antrenament. Efectul este pozitiv, dar modest și neuniform: cel mai consistent câștig apare la *podiumul exact*, unde pădurea aleatoare crește de la 0,333 la 0,375 ($\Delta = +0,042$), iar arborele de la 0,167 la 0,208 ($\Delta = +0,041$); rata de nimerire a podiumului crește pentru arbore ($\Delta = +0,056$) și regresia logistică ($\Delta = +0,014$). Acuratețea rămâne practic neschimbată,

fiind dominată de clasa majoritară (Secțiunea 4.5). Pe câteva metrici efectul este ușor negativ, ceea ce reflectă variabilitatea inerentă a unui singur sezon.

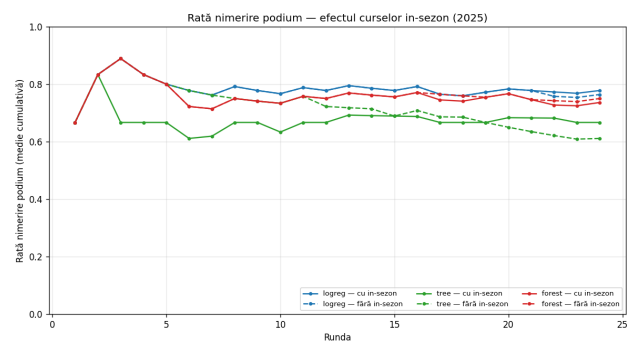
Tabelul 4.5 Performanța medie pe sezonul 2025, cu și fără cursele in-sezon în antrenament.

Model	Regim	Acuratețe	Rată podium	Podium exact	Câștigător
Regresie logistică	fără in-sezon	0,868	0,764	0,375	0,417
	cu in-sezon	0,868	0,778	0,375	0,417
Arbore de decizie	fără in-sezon	0,896	0,611	0,167	0,500
	cu in-sezon	0,887	0,667	0,208	0,417
Random Forest	fără in-sezon	0,921	0,750	0,333	0,500
	cu in-sezon	0,919	0,736	0,375	0,542

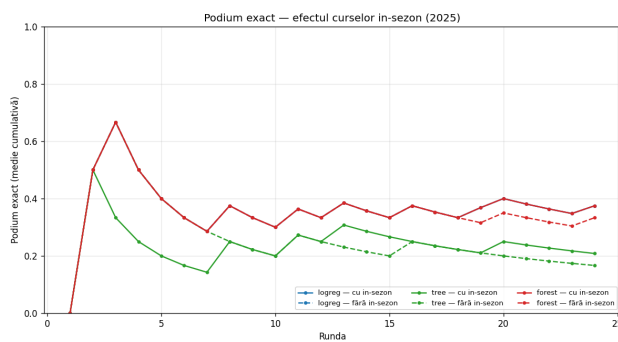
Figura 4.4 prezintă media cumulativă a celor patru metrici pe parcursul sezonului. Cele două regimuri pornesc identic la prima rundă (nu există curse in-sezon anterioare), apoi curbele „cu in-sezon” (linie continuă) se desprind treptat de cele „fără” (linie întreruptă), pe măsură ce modelul acumulează date din sezonul curent.



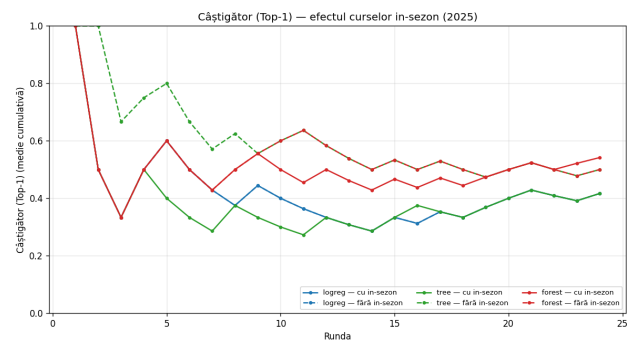
(a) Acuratețe



(b) Rată de nimerire a podiumului



(c) Podium exact



(d) Câștigător (Top-1)

Figura 4.4 Efectul curselor in-sezon pe sezonul 2025: media cumulativă pe runde, cu in-sezon (linie continuă) față de fără (linie întreruptă), pentru cele trei modele.

4.8 Sinteza rezultatelor

Rezultatele experimentale conturează un tablou coerent. Pe ansamblu, pădurea aleatoare este modelul cel mai performant pe trei dintre cele patru metrici, dar avantajul său față de regresia logistică este mic, iar regresia logistică o depășește la acuratețea câștigătorului și se antrenează de aproximativ cinci ori mai repede. Arborele de decizie individual rămâne în mod constant cel mai slab pe metricile de podium, confirmând empiric superioritatea metodelor de ansamblu și a regularizării, discutată teoretic în Capitolul 2.

Studiul evidențiază totodată trei aspecte metodologice. Primul este că acuratețea clasificării este o metrică înșelătoare în prezența dezechilibrului de clase (Secțiunea 4.5): valori de peste 0,90 reprezintă doar o îmbunătățire modestă față de reperul trivial de 85%, motiv pentru care metricile la nivel de cursă sunt cele relevante. Al doilea este că performanța variază considerabil de la un sezon la altul (Secțiunea 4.3), mai ales la nivelul predicției câștigătorului, în funcție de cât de echilibrat este sezonul. Al treilea este că antrenarea incrementală în-sezon aduce un câștig real, dar modest și neuniform, cel mai vizibil la podiumul exact. Aceste concluzii sunt reluate și interpretate în Capitolul 5.

Capitolul 5

Concluzii și dezvoltări ulterioare

5.1 Concluzii generale

Lucrarea de față a abordat problema predicției podiumului unei curse de Formula 1 prin trei algoritmi de învățare automată implementați integral, fără biblioteci de învățare automată, și a realizat un studiu comparativ riguros între aceștia. Toate obiectivele formulate în Capitolul 1 au fost atinse: cei trei algoritmi au fost implementați de la zero peste NumPy, aplicați pe un set de date real (3458 de rezultate din sezoanele 2018–2025) cu o etapă de inginerie a caracteristicilor strict cauzală, și comparați în condiții identice de antrenare și evaluare temporală. Dincolo de rezultatele numerice, valoarea principală a lucrării constă în metodologia riguroasă de evaluare — validare temporală cu separare în antrenare, validare și test pe ani, fără scurgere de informație — și în transparența completă a implementării.

5.2 Gradul de îndeplinire a obiectivelor

Tabelul 5.1 prezintă sintetic gradul de îndeplinire a celor cinci obiective propuse în planul tematic, împreună cu dovada concretă a realizării fiecăruia. Toate obiectivele au fost îndeplinite integral.

5.3 Concluzii privind studiul comparativ

Studiul comparativ, realizat în condiții identice de antrenare și evaluare temporală, a evidențiat următoarele concluzii principale:

- pădurea aleatoare oferă cea mai bună acuratețe (0,903), rată de nimerire a podiumului (0,673) și podium exact (0,253), confirmând avantajul metodelor de ansamblu pe date tabelare;

Tabelul 5.1 Gradul de îndeplinire a obiectivelor propuse.

Obiectiv	Stadiu	Dovadă / rezultat
Implementarea de la zero a trei modele (regresie logistică, arbore, pădure) folosind doar NumPy	Îndeplinit	cele trei module din pachetul <code>f1pred</code> , validate prin teste unitare (Secțiunea 3.13)
Aplicarea predicțiilor pe date reale, cu inginerie a caracteristicilor strict cauzală	Îndeplinit	29 de caracteristici cauzale, garanție anti-leakage testată automat (Secțiunea 3.6)
Studiu comparativ între cei trei algoritmi pe patru metrici	Îndeplinit	Tabelul 4.1 și Figura 4.2
Evaluare cu separare temporală în antrenare, validare și test pe ani	Îndeplinit	validare pe șase sezoane de test (2020–2025), Secțiunea 3.9
Analiza efectului datelor in-sezon	Îndeplinit	comparația cu / fără in-sezon, Secțiunea 4.7

- regresia logistică obține cea mai bună acuratețe a câștigătorului (Top-1, 0,536) și rămâne foarte aproape de pădure pe celelalte metrici, deși se antrenează de aproximativ cinci ori mai repede — fiind, astfel, o alegere simplă, eficientă și competitivă;
- arborele de decizie individual este în mod constant inferior pe metricile de podium, ceea ce confirmă empiric varianța mare a unui arbore unic și avantajul regularizării și al medierii;
- în prezența dezechilibrului de clase (circa 15% exemple pozitive), acuratețea este o metrică înșelătoare, iar evaluarea trebuie să se bazeze pe metrici specifice problemei: rata de nimerire a podiumului, podiumul exact și acuratețea câștigătorului;
- folosirea datelor deja disponibile din sezonul curent (antrenare incrementală in-sezon) aduce îmbunătățiri reale, dar modeste și neuniforme, cele mai vizibile la podiumul exact.

5.4 Contribuții personale

Principalele contribuții ale lucrării sunt: (i) implementarea integrală, de la zero, a celor trei algoritmi de învățare automată, inclusiv a mecanismelor lor interne (coborârea pe gradient, criteriul entropie–câștig de informație, bagging și subspații aleatoare); (ii) proiectarea unei etape de inginerie a caracteristicilor strict cauzale, însoțită de teste automate care garantează absența scurgerii de informație; (iii) construirea unei proceduri de validare temporală cu separare pe ani în antrenare, validare și test; (iv) implementarea manuală a unui set de metrici specifice problemei, calculate la nivel de cursă; și (v) analiza originală a efectului antrenării incrementale in-sezon. Întreaga soluție este expusă printr-o interfață în linie de comandă reproductibilă.

5.5 Dezvoltări ulterioare

Pe termen scurt, soluția poate fi extinsă în mai multe direcții, fără modificarea arhitecturii existente. În primul rând, includerea unor algoritmi suplimentari — mașini cu vectori suport [26] sau metode de tip *gradient boosting* [20], [21] — ar îmbogăți studiul comparativ, modularitatea sistemului permițând adăugarea lor doar prin respectarea interfeței comune de model. În al doilea rând, calibrarea explicită a probabilităților (Secțiunea 2.11), prin scalare Platt sau regresie izotonică, ar putea îmbunătăți interpretarea acestora și acuratețea agregării în clasamentul de campionat. În al treilea rând, ingineria caracteristicilor poate fi rafinată prin includerea unor informații de cronometraj mai fine (ritmul în antrenamentele libere, degradarea anvelopelor, performanța pe tururi rapide), pe care biblioteca FastFl le pune la dispoziție. În al patrulea rând, optimizarea automată a hiperparametrilor — realizată strict pe sezonul de validare, pentru a evita scurgerea de informație — ar putea ridica performanța fiecărui model. Nu în ultimul rând, paralelizarea antrenării arborilor din pădure ar reduce costul computațional, singurul dezavantaj notabil al celui mai performant model.

Pe termen mai lung, abordarea ar putea fi extinsă de la predicția podiumului la predicția poziției complete a fiecărui pilot, printr-o reformulare ca problemă de ordonare propriu-zisă (eng. *learning to rank*) [9], ceea ce ar exploata și mai bine structura competiției.

Bibliografie

- [1] C. R. Harris et al., „Array Programming with NumPy,” *Nature*, vol. 585, nr. 7825, pp. 357–362, 2020. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)
- [2] F. Pedregosa et al., „Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] H. He și E. A. Garcia, „Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, nr. 9, pp. 1263–1284, 2009. DOI: [10.1109/TKDE.2008.239](https://doi.org/10.1109/TKDE.2008.239)
- [4] T. Saito și M. Rehmsmeier, „The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLoS ONE*, vol. 10, nr. 3, e0118432, 2015. DOI: [10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432)
- [5] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.
- [6] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [7] T. Hastie, R. Tibshirani și J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, a 2-a ed. New York: Springer, 2009.
- [8] G. James, D. Witten, T. Hastie și R. Tibshirani, *An Introduction to Statistical Learning with Applications in R*. New York: Springer, 2013.
- [9] T.-Y. Liu, „Learning to Rank for Information Retrieval,” *Foundations and Trends in Information Retrieval*, vol. 3, nr. 3, pp. 225–331, 2009. DOI: [10.1561/15000000016](https://doi.org/10.1561/15000000016)
- [10] S. Geman, E. Bienenstock și R. Doursat, „Neural Networks and the Bias/Variance Dilemma,” *Neural Computation*, vol. 4, nr. 1, pp. 1–58, 1992. DOI: [10.1162/neco.1992.4.1.1](https://doi.org/10.1162/neco.1992.4.1.1)
- [11] T. Fawcett, „An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, nr. 8, pp. 861–874, 2006. DOI: [10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010)
- [12] N. V. Chawla, K. W. Bowyer, L. O. Hall și W. P. Kegelmeyer, „SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. DOI: [10.1613/jair.953](https://doi.org/10.1613/jair.953)
- [13] S. Boyd și L. Vandenberghe, *Convex Optimization*. Cambridge: Cambridge University Press, 2004.
- [14] L. Breiman, J. H. Friedman, R. A. Olshen și C. J. Stone, *Classification and Regression Trees*. Monterey, CA: Wadsworth și Brooks/Cole, 1984.

- [15] J. R. Quinlan, „Induction of Decision Trees,” *Machine Learning*, vol. 1, nr. 1, pp. 81–106, 1986. DOI: 10.1007/BF00116251
- [16] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993.
- [17] T. G. Dietterich, „Ensemble Methods in Machine Learning,” în *Multiple Classifier Systems*, ser. Lecture Notes in Computer Science, vol. 1857, Berlin, Heidelberg: Springer, 2000, pp. 1–15. DOI: 10.1007/3-540-45014-9_1
- [18] L. Breiman, „Random Forests,” *Machine Learning*, vol. 45, nr. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324
- [19] Y. Freund și R. E. Schapire, „A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *Journal of Computer and System Sciences*, vol. 55, nr. 1, pp. 119–139, 1997. DOI: 10.1006/jcss.1997.1504
- [20] J. H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics*, vol. 29, nr. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451
- [21] T. Chen și C. Guestrin, „XGBoost: A Scalable Tree Boosting System,” în *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785
- [22] R. Kohavi, „A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” în *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 1995, pp. 1137–1143.
- [23] C. Bergmeir și J. M. Benítez, „On the Use of Cross-Validation for Time Series Predictor Evaluation,” *Information Sciences*, vol. 191, pp. 192–213, 2012. DOI: 10.1016/j.ins.2011.12.028
- [24] A. Niculescu-Mizil și R. Caruana, „Predicting Good Probabilities with Supervised Learning,” în *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430
- [25] R. P. Bunker și F. Thabtah, „A Machine Learning Framework for Sport Result Prediction,” *Applied Computing and Informatics*, vol. 15, nr. 1, pp. 27–33, 2019. DOI: 10.1016/j.aci.2017.09.005
- [26] C. Cortes și V. Vapnik, „Support-Vector Networks,” *Machine Learning*, vol. 20, nr. 3, pp. 273–297, 1995. DOI: 10.1007/BF00994018
- [27] W. McKinney, „Data Structures for Statistical Computing in Python,” în *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a
- [28] J. D. Hunter, „Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, nr. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55
- [29] P. Schaefer și FastF1 contributors, *FastF1: Access to Formula 1 Timing Data and Telemetry*, <https://github.com/the0ehrly/Fast-F1>, Accesat: 2026-06-05, 2024.